InvestoGenie
External AI Review Pack

Architecture, capabilities, deployed status, strategy contracts, ML methodology, operations, security and mobile guidance

Prepared 14 November 2026 | Deployed revision f33f256

Important: tests and documentation do not prove profitability or continuing provider health. Review point-in-time evidence and fail-closed behavior.

Table of contents

Table of contents 2

Source document: AI_REVIEW_HANDOFF.md . . . 8

InvestoGenie external AI review handoff 8

Review objective . . . 8

Read these files first . . . 8

Current production boundary . . . 8

Calculation ownership . . . 9

High-impact invariants to review . . . 9

Strategy lifecycle versions . . . 9

Validation already performed . . . 10

Known open evidence and enhancement areas . . . 10

Required format for reviewer findings . . . 10

Useful commands . . . 11

Source document: DOCUMENTATION_INDEX.md . . . 12

InvestoGenie documentation index 12

Authority order . . . 12

Source document: README.md . . . 14

InvestoGenie 14

Workspaces . . . 14

Execution and accounting . . . 14

Stack . . . 15

Local setup . . . 15

Jobs and providers . . . 15

Verification and deployment . . . 15

Source document: ARCHITECTURE.md . . . 17

InvestoGenie Architecture 17

System flow . . . 17

Repository boundaries . . . 17
Calculations and candidate flow . . . 18
Immutable plans and ledger writes . . . 18
News and fund data . . . 19
Schedulers and providers . . . 19
Mobile and secrets . . . 19
ML foundation . . . 19
Deployment and verification . . . 20
Source document: CAPABILITIES.md . . . 21
InvestoGenie Capabilities **21**
Market research . . . 21
Armed Swing and execution plans . . . 21
Trade Ledger and broker behavior . . . 22
Funds, credentials and automation . . . 22
Mobile . . . 22
ML and operational limits . . . 23
Source document: STATUS.md . . . 24
InvestoGenie Status **24**
Implemented in the reviewed source . . . 24
Latest schema additions . . . 24
Verified production baseline - 14 November 2026 . . . 25
Strategy review and changes - 13-14 November 2026 . . . 25
Current behavior that matters operationally . . . 26
Verification available in the repository . . . 26
Remaining work and runtime checks . . . 27
Source document: DEPLOYMENT.md . . . 28
InvestoGenie Deployment Targets **28**
Configuration isolation . . . 28

Promotion workflow . . . 28
Migration and release limits . . . 29
Source document: AGENTS.md . . . 30
This is NOT the Next.js you know **30**
InvestoGenie Agent Instructions **30**
Start Here . . . 30
Context Efficiency . . . 30
Repository Boundaries . . . 31
Architecture Rules . . . 31
Protected Financial Logic . . . 32
Development Workflow . . . 32
Validation . . . 33
Database Safety . . . 33
Secrets and Security . . . 33
Git Safety . . . 34
Deployment Safety . . . 34
Mobile Safety . . . 34
Communication . . . 35
Source document: README-screener.md . . . 36
Stock Screener **36**
Data flow . . . 36
Setup and refresh . . . 36
Universes and saved screens . . . 37
Natural-language filters . . . 37
Validation . . . 37
Source document: docs/swing-strategies.md . . . 38
Swing, Strong Swing and Armed Strong Swing: rules and review **38**
1. Swing Candidates (/terminal/[market]/screener) . . . 38

Base classifier . . . 38
Legendary tags . . . 38
After purchase . . . 39
2. Strong Swing (/terminal/[market]/strong-swing) . . . 39
3. Armed Strong Swing (/terminal/[market]/armed-swing) . . . 40
4. Review findings and how each was closed . . . 40
Left as designed . . . 41
Production evidence and remaining observation . . . 41
Source document: docs/ml-methodology.md . . . 42
InvestoGenie ML Methodology - Production Contract v1 42
Implemented versus specified . . . 42
Purpose and boundary . . . 42
Frozen production policies . . . 42
Canonical data . . . 43
Label semantics . . . 43
Model lifecycle and serving contract . . . 43
Registered promotion gates . . . 43
Monitoring and failure-behavior contract . . . 44
Current implementation state . . . 44
Verification . . . 44
Source document: docs/EMAIL_SETUP.md . . . 45
Email Digest Setup 45
Configuration . . . 45
Scheduling . . . 45
Manual and external invocation . . . 45
Monitoring and troubleshooting . . . 46
Source document: docs/QUICK_START_CREDENTIALS.md . . . 47
Quick Start: Credentials and Email Digest 47

1. Prepare the application . . . 47
2. Save SMTP credentials . . . 47
3. Enable the digest . . . 47
4. Optional AI/news setup . . . 47
5. Verify . . . 47
Source document: docs/SECURE_CREDENTIALS.md . . . 48
Credential Storage **48**
Implementation . . . 48
Credential domains . . . 48
Setup and recovery . . . 48
Settings and server access . . . 48
Fallbacks are caller-specific . . . 49
Troubleshooting . . . 49
Source document: deploy/local/README.md . . . 50
Local macOS deployment **50**
First setup . . . 50
Always-on personal service . . . 50
Gmail mutual-fund disclosures . . . 51
Private access with Tailscale . . . 51
Local password recovery . . . 51
Local run modes . . . 51
Relationship to Oracle . . . 52
Source document: deploy/oracle/README.md . . . 53
InvestoGenie on AWS Lightsail or Oracle-compatible Ubuntu **53**
1. Create the VM . . . 53
2. Publish the application revision . . . 53
3. Bootstrap Ubuntu . . . 53
4. Transfer the current database . . . 54

5. Build, start, and enable HTTPS . . . 54
6. Releases and backups . . . 54
Operating notes . . . 55
ICICI Breeze live market data **55**
Schema and release verification . . . 55
Source document: mobile/README.md . . . 57
InvestoGenie Mobile **57**
Setup . . . 57
Implemented surfaces . . . 57
Local release builds . . . 57
Trade logging and security . . . 58
Local builds and configuration . . . 58
Server API . . . 58
Source document: mobile/PRIVACY.md . . . 59
InvestoGenie Mobile Privacy **59**

Source document: AI_REVIEW_HANDOFF.md

InvestoGenie external AI review handoff

Prepared 14 November 2026 against deployed revision `f33f256`.

Review objective

Review InvestoGenie for correctness, safety, data integrity, timing, coverage and maintainability. Separate defects from strategy preferences. Do not recommend loosening a fail-closed gate merely to produce more candidates. Every proposed financial-logic change should state the expected benefit, failure mode, evidence required, affected policy/model version and replay/forward-test plan.

InvestoGenie is a self-hosted India/US market-research and trade-ledger application. It is not an exchange or broker. Most actions record plans or ledger events; Breeze auto-buy is a separately enabled, capped India cash-limit path. There is no automatic selling.

Read these files first

Order	File	Purpose
1	`AGENTS.md`	Repository rules, protected financial logic and deployment safety
2	`ARCHITECTURE.md`	Runtime boundaries, data flow and ownership
3	`docs/swing-strategies.md`	Exact Swing, Strong Swing, Armed and named-strategy rules
4	`docs/ml-methodology.md`	ML production contract and what is not implemented
5	`STATUS.md`	Dated source and production evidence
6	`CAPABILITIES.md`	Implemented product behavior
7	`DEPLOYMENT.md`	Promotion, migration and service boundaries

Source code wins when documentation disagrees.

Current production boundary

Item	State at review handoff
Revision	`f33f256` on `main`, deployed to AWS Lightsail
Web/Breeze services	Active at the recorded check
Public health	HTTPS `/login` returned HTTP 200
Schema	Migrations through 0077; production schema verification passed
India results calendar	2,943/2,945 active NSE assets verified (99.93%)
India Armed volume curve	`in-median-2026-09-16-2026-11-13-40s`; 1,425 complete asset-sessions, 40 sessions
US Armed	Fail-closed: no US execution-depth/intraday provider
Strong Swing forward evidence	Five genuine scheduled v4 captures still required
ML serving	No validated serving assignment established

The India curve uses Breeze historical one-minute candles. A short special session was excluded because it did not meet the full-session completeness rule. Breeze does not supply US equities.

Calculation ownership

Domain	Authority
Strong Swing and Momentum engines	Python under `pipelines/strategy_math/`; TypeScript is input/loading or a test oracle
Trend Continuation	Python persistence and execution gates
Base Swing and named-strategy tags	TypeScript under `lib/analytics/`
Armed Strong Swing	TypeScript `lib/analytics/armedSwing.ts` plus database loader
Ledger projection/follow-up	Server TypeScript `lib/swingTradeProjection.ts`, `lib/swingTradeLedger.ts`
ML features and labels	Python only
Web/mobile	Render server-produced results; must not independently calculate trades

High-impact invariants to review

1. Calculations that need a close or full-day volume use completed sessions; live quotes are a separate execution overlay.
2. Strong Swing independently recomputes ATR, breakout-bar evidence, entry, stop, target, invalidation and results blackout.
3. Purchase actions load immutable server plans and reject stale versions, mismatched assets/statuses and out-of-band prices.
4. India actionability requires fresh Breeze execution evidence where specified; missing evidence blocks rather than falls back.
5. Armed volume pace uses an active market-specific curve backed by at least 40 complete sessions.
6. Results safety requires an explicit upcoming result date or authoritative coverage spanning the entire three-session blackout window.
7. Long stops only tighten upward; short stops only tighten downward.
8. EMA/SMA strategy exits require a completed-close crossing and do not replace catastrophic live broker protection.
9. Simons moving targets are accepted only on the profitable side of entry for the trade direction.
10. Browser, mobile and email notifications do not create eligibility or place orders.

Strategy lifecycle versions

Contract	Version
Strong Swing model	`strong-swing-v4-breakout-bar`
Strong Swing feature contract	`strong-swing-features-v2`
Strong Swing label	`strong-swing-first-touch-v1`
Legendary strategies	`legendary-v2-2026-11-14`
Armed Swing	`armed-v3-2026-11-14`
India volume curve	`in-median-2026-09-16-2026-11-13-40s`

Any threshold, feature, label, entry, trailing, inclusion or calibration change needs a new version and fresh evidence.

Validation already performed

Check	Result
TypeScript/Vitest	573 passed
Pipeline Python unittest discovery	91 passed
Worker Python unittest discovery	77 passed
TypeScript typecheck	Passed
ESLint	Passed
Next.js production build	Passed
Migration 0077 and schema verifier	Passed in production
Production services and HTTPS login	Passed at the recorded check

Tests prove coded behavior, not profitability or continuing provider health.

Known open evidence and enhancement areas

Priority	Area	Current boundary	Suggested review question
High	Strong Swing forward evidence	Fewer than five post-v4 scheduled captures	Are candidate rate, precision, expectancy and calibration acceptable without changing gates post hoc?
High	US Armed	No US depth/circuit-halt/intraday source	Which licensed provider supplies point-in-time bid/ask, halts and intraday volume with auditable timestamps?
High	ML lifecycle	Schema/features/labels exist; training, calibration, serving and drift automation incomplete	What is the smallest production implementation that satisfies `docs/ml-methodology.md` without affecting eligibility?
High	Auto-buy safety	India buy-only, explicitly enabled and capped	Are idempotency, unknown-order reservation, broker reconciliation and regime sizing complete under retries?
Medium	Results calendar	India coverage high but provider health is external	How should source disagreement and late schedule changes be monitored and auto-suspended?
Medium	Curve maintenance	India curve is active; scheduler adds sessions	Should curves be segmented by liquidity bucket or session type without overfitting?
Medium	Ledger scale-outs	Planned quantities are advisory/user-confirmed	Should the UI add explicit fill reminders and reconciliation without sending sell orders?
Medium	Special sessions	Short sessions are excluded from regular curves	Should special-session curves be a separately versioned policy rather than treated as regular sessions?
Medium	Email delivery	Per-user send time/last-sent enforcement remains incomplete	Add durable exactly-once delivery keys and per-user schedules?
Low	Local metadata	`.claude/launch.json` is machine-local and uncommitted	Keep it excluded from release review.

Required format for reviewer findings

For every finding provide:

Field	Required content
Severity	P0-P3
File/line	Exact current source location
Broken invariant	Which documented contract is violated
Reproduction	Minimal deterministic fixture/query
User impact	Missed candidate, false candidate, wrong level, stale data, unsafe action or audit gap
Proposed fix	Smallest server-authoritative change
Tests	Unit, parity, replay and/or forward evidence required
Version impact	Whether model/policy/feature/label version must change
Rollout	Shadow, fail-closed behavior, migration and rollback

Avoid claims based only on screenshots or hindsight. A chart that later rose does not prove it met point-in-time liquidity, freshness, volume, results, spread, regime and entry-geometry rules at the earlier decision time.

Useful commands

```
npm test
npm run lint
npm run build
PYTHONPATH=pipelines:workers .venv/bin/python -m unittest discover -s pipelines -p 'test_*.py'
PYTHONPATH=pipelines:workers .venv/bin/python -m unittest discover -s workers -p 'test_*.py'
npm run audit:strategy-readiness
```

The readiness audit is expected to remain fail-closed until the required genuine Strong Swing captures exist and will continue to report US Armed missing until a proper US source is configured.

Source document: DOCUMENTATION_INDEX.md

InvestoGenie documentation index

Reconciled 14 November 2026 through deployed revision `f33f256`.

Use AI_REVIEW_HANDOFF.md (AI_REVIEW_HANDOFF.md) when handing the repository to another AI reviewer. The index below explains the scope of every Markdown document; generated machine-local Claude context is not part of the product contract.

Document	Scope
`README.md`	Product entry point, routes, stack, setup and release commands
`AI_REVIEW_HANDOFF.md`	External AI review brief, invariants, evidence and requested finding format
`ARCHITECTURE.md`	Runtime, repository and calculation ownership boundaries
`CAPABILITIES.md`	Implemented application behavior
`STATUS.md`	Dated source/production verification and known gaps
`DEPLOYMENT.md`	Environment separation and production promotion workflow
`AGENTS.md`	Mandatory coding, testing, safety and deployment instructions for coding agents
`CLAUDE.md`	Claude-specific pointer and machine-generated local session context; not an application contract
`README-screener.md`	Stock Screener filters, data and limits
`docs/swing-strategies.md`	Exact Swing, Strong Swing, Armed and named-strategy rules
`docs/ml-methodology.md`	ML production lifecycle contract and frozen semantics
`docs/EMAIL_SETUP.md`	Digest configuration and operational limits
`docs/QUICK_START_CREDENTIALS.md`	Short credential setup guide
`docs/SECURE_CREDENTIALS.md`	Credential encryption and provider setup
`deploy/local/README.md`	macOS local service installation
`deploy/oracle/README.md`	AWS Lightsail/Ubuntu bootstrap and releases
`mobile/README.md`	Mobile client development and distribution
`mobile/PRIVACY.md`	Mobile data and notification privacy boundary

Authority order

1. Current source code and database migrations.
2. `AGENTS.md` for repository workflow and protected financial logic.
3. `docs/swing-strategies.md` and `docs/ml-methodology.md` for versioned financial contracts.
4. `ARCHITECTURE.md`, `CAPABILITIES.md` and `STATUS.md` for system behavior and dated evidence.
5. Setup and product guides.

Documentation cannot establish continuing provider freshness, profitability, a deployed mobile binary, or an active ML serving assignment. Those require runtime evidence.

Source document: README.md

InvestoGenie

InvestoGenie is a self-hosted market research and trade-management application for Indian and US equities. Next.js serves the web terminal and mobile APIs; PostgreSQL stores market data, user portfolios, frozen trade plans and operational history.

_Code and production baseline reviewed 14 November 2026 through deployed revision `f33f256`. See STATUS.md (STATUS.md) for the dated verification boundary and AI review handoff (AI_REVIEW_HANDOFF.md) for an external-review guide._

Workspaces

Route	Purpose
`/markets/in`, `/markets/us`	Market overview and charts
`/terminal/[market]`	India (`in`) or US (`us`) terminal
`/terminal/[market]/stocks`	Fundamental and price-action Stock Screener, saved screens, natural-language queries and exports
`/terminal/[market]/screener`	Buy-side Swing Candidates with all current setups, strategy tags and execution-state notes
`/terminal/[market]/strong-swing`	Strong Swing, legacy/V2 Momentum Ignition and Python Trend Continuation observation
`/terminal/[market]/armed-swing`	Earlier pre-breakout Armed Strong Swing plans
`/terminal/[market]/news-swing`	Strong Swing with source-linked news and AI assessment
`/terminal/[market]/trade-ledger`	Purchases, pending buy-stops, partial fills/exits, reconciliation and returns
`/terminal/[market]/long-term`	Multi-year fundamentals and investor-inspired rankings
`/terminal/[market]/probability`	21-session probability forecasts
`/terminal/[market]/adaptive-allocation`	Cost-aware two-asset backtests and saved strategies
`/terminal/[market]/forward-test`	Forward measurement of recorded signals
`/terminal/in/cas`, `/portfolio/fund-mapping`	CAS import, Gmail disclosures, AMFI mapping and fund overlap
`/data/health`, `/admin/sync`	Freshness, coverage and ingestion health
`/settings`, `/help`	Account/provider configuration and product guidance

Execution and accounting

Analytics run on the server. Strong Swing and Momentum purchase flows use stored execution plans; Armed Swing stores its own immutable trigger, limit, stop, target and gate snapshot. Pending buy-stops are ledger records: creating one sends no broker order and creates no position or P&L. Confirmed partial fills create separate purchased trades while preserving the unfilled remainder.

Armed setups require verified results-calendar coverage, a market-specific 40-session intraday volume curve and live execution data. Missing evidence blocks actionable states. India execution quotes require `BREEZE_LIVE`; a fallback quote alone cannot authorize an Armed entry. The active India curve at the reviewed production boundary contains 1,425 complete Breeze asset-sessions across 40 sessions.

The ledger separates cumulative purchase turnover from external capital employed after recycling retained sale proceeds. ROI uses capital employed; XIRR uses dated contributions and terminal cash plus open-position value. Broker-reconciled acquisition/proceeds/P&L can replace gross values.

Breeze automatic buying is a separate, disabled-by-default, explicitly enabled path for capped NSE cash limit buys from Strong Swing and saved Adaptive Allocation strategies. It does not automatically sell or rebalance. Research projections and notification delivery do not place orders.

Stack

Layer	Repository dependency/configuration
Web	Next.js 16.4.2, React 19.2.4, TypeScript
UI	Tailwind CSS 4, Lightweight Charts, Three.js
Database	PostgreSQL through pg
Mobile	Expo ~57.0.24, React Native 0.86.3, React 19.2.3
Jobs	Node scheduler and Python pipelines/workers
Credentials	AES-256-GCM per-user encrypted secrets
Deployment package	macOS launchd or Ubuntu systemd/Nginx; Linux scripts support AWS/Oracle hosts

Local setup

The supplied local commands target macOS/Linux. Use a current compatible Node.js runtime, Python 3 and PostgreSQL; several package scripts expect `.venv/bin/python`.

```
cp .env.example .env.local
npm ci
python3 -m venv .venv
.venv/bin/pip install -r pipelines/requirements.txt -r workers/requirements.txt
```

Create the database named in `DATABASE_URL`. Set distinct random `SESSION_SECRET`, `CRON_SECRET` and `CREDENTIAL_ENCRYPTION_KEY` values. Set one nonempty `NEXT_PUBLIC_APP_URL` (the example currently contains two declarations, so remove or replace the blank one). For a fresh database, apply the ordered SQL migrations before starting; see local setup (deploy/local/README.md).

```
npm run deploy:check:local
npm run dev
```

Open `http://localhost:3000`. Both `npm run dev` and `npm start` run the scheduler wrapper. `dev:next` and `start:next` start only Next.js.

Jobs and providers

The wrapper coordinates market-hours quotes, EOD ingestion, history repair, fundamentals, macro, AMFI, scans, forward tests, news, email, mobile alerts, Gmail processing, results calendars and scheduled ML snapshots. Breeze supplies live India data; exchange Bhavcopy, Yahoo/Google and FRED support the other ingestion paths. News providers include Marketaux, GNews, NewsAPI and Alpha Vantage.

Momentum scans run during market hours even when an account has no registered push device. India actionability uses a separately verified current-session Breeze OHLCV/volume bar; completed history remains immutable during the session. Candidate state changes are retained for audit, while push delivery still requires an enabled mobile token.

The ML implementation is a Python feature/label foundation and lifecycle schema. The repository does not include a complete training, promotion or inference pipeline. See ML methodology (docs/ml-methodology.md).

Verification and deployment

```
npm test
```

```
npm run lint
npm run build
```

Database checks need a configured PostgreSQL instance. Linux releases use `deploy/oracle/deploy-release.sh`; it discovers and applies migrations from 0033 onward in filename order and assumes the earlier schema already exists. After updating the checkout, it re-executes the checked-out release script once so same-release migration additions are honored. It runs dependency audit/build/schema checks, but does not run the test suite.

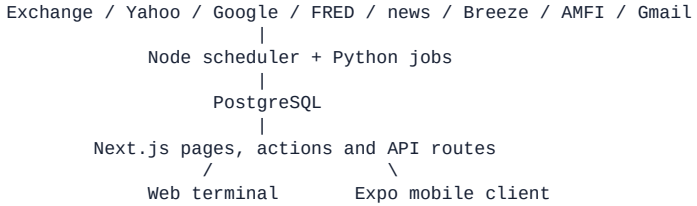
See the documentation index (DOCUMENTATION_INDEX.md), AI review handoff (AI_REVIEW_HANDOFF.md), architecture (ARCHITECTURE.md), capabilities (CAPABILITIES.md), status (STATUS.md), deployment (DEPLOYMENT.md), Stock Screener (README-screener.md), email setup (docs/EMAIL_SETUP.md), and mobile (mobile/README.md).

Source document: ARCHITECTURE.md

InvestoGenie Architecture

Source and production baseline reviewed 14 November 2026 through deployed revision `f33f256`.

System flow



The web stack is Next.js 16.4.2 App Router, React 19.2.4 and TypeScript. PostgreSQL is accessed through `pg`. Tailwind CSS 4, Lightweight Charts and Three.js support presentation. Mobile uses Expo ~57.0.24 / React Native 0.86.3; financial analytics remain server-owned.

Repository boundaries

Path	Responsibility
`app/`	Pages, server actions, cron handlers and mobile APIs
`components/`	Web presentation
`lib/analytics/`	Signal, ranking, risk and strategy calculations
`lib/screener/`	Fundamental/price-action snapshot and filter engine
`lib/news/`	Provider aggregation, evidence and classification
`lib/cas/`, `lib/funds/`, `lib/gmail/`	Portfolio parsing, identifiers, snapshots and disclosure processing
`lib/breeze/`	Broker integration and execution safety
`lib/backfill/`	Persistent OHLCV repair
`lib/mobile*.ts`	Sessions, ledger API support and notifications
`pipelines/`, `workers/`	Python ingestion and broker workers
`pipelines/ml/`	Deterministic ML feature collection and labels
`scripts/`	Scheduling, ingestion and operational checks
`db/migrations/`	Ordered schema through 0077
`mobile/`	Isolated React Native client
`deploy/`	macOS and Ubuntu service/deployment packages

The README route table (README.md#workspaces) maps each workspace. In particular, `stocks` is the filter/NL Stock Screener, `screener` is ordinary Swing Candidates, and `armed-swing` is the earlier Armed workflow.

Calculations and candidate flow

`swingClassifier.ts` and `legendaryStrategies.ts` produce the base technical setups persisted in `swing_signals`. The four strategies displayed on the Strong Swing page are Python-authoritative: `pipelines/strategy_math_worker.py` batch-calculates Strong Swing and both Momentum engines, while `pipelines/trend_continuation.py` owns the persisted Trend Continuation signal and the batch worker owns its live execution gates. TypeScript loaders collect inputs, validate the returned batch and render or persist it; they do not calculate strategy statuses, scores, rankings or trade levels. News & AI adds bounded source evidence to existing setups.

Momentum Ignition legacy and V2 remain separate models but share the Python batch boundary, which also applies the V2 timing veto to legacy execution. Armed Swing is a separate screen and still has a TypeScript assessment in `lib/analytics/armedSwing.ts` and database loader in `lib/armedSwing.ts`; it is outside the Strong Swing-page migration. The loader scans market-relative strength and near-pivot candidates, then assesses at most 250 shortlisted stocks. It excludes the in-progress bar from base construction and uses live volume only for breakout pace.

Trend Continuation is a separate Python-owned observation engine (`pipelines/trend_continuation.py`). It detects persistent 3-5 session impulses, validates continuation or extension risk, and stores versioned results in `trend_continuation_signals`. Its quote freshness, spread, circuit room, results blackout, price-band and plan-freshness gates are also evaluated in Python by `pipelines/strategy_math/trend_execution.py`. Observation-only candidates have no auto-buy path.

Momentum discovery likewise separates completed history from the current-session bar. During an open India session, entry-ready classification requires a fresh `BREEZE_LIVE` quote plus the matching Breeze intraday OHLCV/volume row; otherwise the setup remains visible but execution-blocked. Scheduled candidate scans run for every account independently of Expo push-token registration, persist status transitions in `candidate_signal_observations`, and refresh `live_market_targets`. The Breeze worker reconciles its bounded subscription set every two minutes so newly discovered candidates receive live bars without a service restart.

Armed actionability depends on live execution quotes, bid/ask, circuit bands, liquidity, structure, relative-strength leadership, market regime, a market-specific time-of-day volume curve and verified results-calendar coverage. India requires a fresh `BREEZE_LIVE` quote. Missing evidence produces unverified or blocked states. Results within three trading sessions block entries. The curve builder accepts only complete regular sessions and needs at least 40 sessions; India production has an active 40-session Breeze curve, while US Armed remains fail-closed without a US depth/intraday source.

`pipelines/corporate_results_sync.py` imports the official NSE event calendar and supplements dates through Yahoo. Authoritative negative coverage requires a complete same-day window covering the required sessions; an empty Yahoo result alone does not verify safety.

Long-term, probability and Universal Portfolio calculations live in their own analytics modules. Adaptive Allocation is a research/backtest surface with saved strategies that can separately supply the explicitly enabled Breeze buy-only workflow.

Immutable plans and ledger writes

`lib/armedSwingPlans.ts` persists immutable Armed calculations, including status, levels, ATR, gates and a five-minute capture bucket. Actions load plans by ID and user ownership rather than trusting browser-submitted levels.

- Pending Armed buy-stops require a same-session `ARMED` plan captured within 30 minutes.
- Direct Armed purchases require a matching `BUY_ZONE` plan and purchase date, with a fill inside the frozen trigger/limit band subject to tolerance.
- `lib/swingExecutionPlans.ts` persists Strong Swing/Momentum plans. Purchase validation checks matching asset/market/strategy, actionable state, current session, capture age and price range.
- Edits cannot move protected purchases below their frozen entry/trigger.

Pending-order actions in `app/terminal/[market]/trade-ledger/pendingOrderActions.ts` record orders without placing broker instructions. Default validity is seven calendar days. Fill recording locks the order row, validates remaining quantity/price/date, inserts a purchased ledger trade and fill audit, then updates the remainder atomically. Each partial fill is a separate trade; cancellation preserves filled trades.

Ledger projection/risk calculations are shared through `lib/swingTradeProjection.ts` and `lib/swingTradeLedger.ts`. Direction-sensitive protection ensures long stops only rise and short stops only fall. Named strategies use completed-close exit lines, while live catastrophic protection remains separate. Qullamaggie, Minervini, Darvas and PTJ plans schedule a 50% first-target scale-out; Simons schedules the full mean-reversion allocation. Cash-flow accounting distinguishes turnover from external capital after retained proceeds are reused. ROI and XIRR use that capital model. Broker reconciliation and explicitly enabled capped NSE cash limit buying are separate paths; no automatic selling/rebalancing is implemented.

News and fund data

News providers are queried independently and successful results merged. Watermarks, canonical deduplication and corroboration preserve provenance. Priority starts with open ledger trades, then Strong Swing, then broader Swing. Evidence quality and stock specificity constrain risk assessment; broad or unverified stories are cautionary.

CAS holdings use user/ISIN/folio identities. Explicit mappings and AMFI identifiers link them to AMC snapshots for look-through and overlap. Gmail uses read-only OAuth and encrypted tokens; enabled auto-import processes suitable CAS and disclosures, while ambiguous matches remain reviewable. Attachment bytes and email bodies are not retained as document storage. Revoked grants set `needs_reconnect`; scan failures record the latest error and timestamp.

Schedulers and providers

`scripts/run-with-nse-sync.mjs` supervises Next.js and jobs under `dev`/`start`. Bare Next.js scripts skip this wrapper.

Schedules cover market-hours priority quotes, EOD Bhavcopy/history, repair queues, fundamentals, macro, AMFI, scans, forward tests, news, Gmail, mobile alerts, email and ML snapshots. Corporate results run at 07:00/17:00 IST plus startup work. Breeze can be embedded or supervised separately.

India live quotes prefer Breeze, with Yahoo/Google/exchange fallback elsewhere. Bhavcopy maintains EOD history; US recurring history uses Yahoo. The optional Tiingo history module is separate from that recurring Python path. Market-calendar helpers support session-aware freshness.

Email defaults to 07:00 IST with wrapper catch-up and retry limits. The actual digest uses per-user encrypted SMTP and India Strong Swing/Probability results. Stored user send times and last-sent timestamps are not enforced by the sender's enabled-user query; partial retries can duplicate successful deliveries.

Mobile and secrets

Mobile APIs are under `/api/v1/mobile/`: login/logout, me, Strong Swing, News & AI, ledger and sale mutations, asset search, candles and push registration. Random 256-bit device tokens are held in SecureStore; the server stores SHA-256 hashes with 30-day expiry and revocation. Biometrics guard restored sessions when available.

Expo receives push tokens and notification text. Ledger alerts are generic; candidate alerts contain ticker, engine and readiness. Candidate details exclude price, quantity and portfolio/broker values. Notifications do not place orders.

App metadata is 0.1.1, Android version code 2 and iOS build 2; `mobile/package.json` remains 0.1.0. The documented local build path is Expo Prebuild then Gradle/Xcode. EAS scripts/configuration remain in source, and release validation checks EAS profile URLs; it does not prove a locally built binary uses the right URL.

Web sessions use signed HTTP-only cookies. Credentials use AES-256-GCM with a scrypt-derived key and random IV; `getUserCredentials()` returns metadata/presence flags. Server-only resolvers decrypt secrets when needed. Cron callers send the configured bearer secret; handlers must be reviewed individually rather than assumed to share identical checks.

ML foundation

Migration 0051 defines snapshots, labels, models, promotion evidence, serving pointers and monitoring records. Node captures canonical Strong Swing candidates and invokes Python feature collection. Python implements features and first-touch labels with golden vectors.

Training, calibration, inference, model promotion and drift-response automation are specified in the methodology contract (docs/ml-methodology.md), but complete runtime pipelines are not present. Schema existence does not establish an active model or monitoring service.

Deployment and verification

Ubuntu packages use systemd, Nginx, external environment configuration and private PostgreSQL. The historical `deploy/oracle/` path also supports AWS hosts. macOS uses launchd and optional Tailscale access.

The release script fetches main, then re-executes the checked-out script once so the current release logic is used. It installs/audits/builds/prunes Node dependencies, installs Python requirements, discovers and applies migrations 0033 onward in filename order, verifies required schema including migration 0077 market-volume curves, installs service units, restarts and checks login. It assumes the earlier schema and does not execute lint/tests.

Use `npm test`, `npm run lint`, `npm run build` and deployment checks before promotion. Mobile has separate typecheck/release validation; Python suites need separate execution. Runtime coverage, installed builds and production revision require host/database inspection.

Heavy jobs share the web service process tree unless separately managed. Provider limits, missing execution depth, incomplete statements/fund mappings, Expo relay dependence and unfinished ML lifecycle runtime remain operational boundaries.

Source document: CAPABILITIES.md

InvestoGenie Capabilities

Code and production baseline reviewed 14 November 2026 through deployed revision `f33f256`. Implemented means present in source; provider access, continuing data coverage and installed mobile binaries still require runtime checks.

Market research

Surface	Implemented behavior	Main sources
Market overview	India/US quotes, breadth and charts	`app/markets/`, `components/market-overview/`
Stock Screener	Whitelisted AND filters, presets, universes, saved screens, CSV/Excel exports and natural-language filters	`lib/screener/`, `lib/screener-actions.ts`
Swing Candidates	Buy-side setups with strategy tags, breakout-day and extended/void entry states, and frozen levels	`lib/screener.ts`, market screener page
Strong Swing	Python-authoritative confirmation, liquidity, market, results-blackout and execution gates, trade levels and deterministic ranking	`pipelines/strategy_math/strong_swing.py`, `lib/strongSwing.ts`
Momentum Ignition	Python-authoritative legacy and V2 discovery with completed-session bases, separately verified live-session input and explicit execution blocking; V2 includes Momentum Ready, First Thrust, Pre-Ignition, Retest, Late Risk and Failed states	`pipelines/strategy_math/momentum.py`, `lib/momentumIgnition*.ts`
Trend Continuation	Python-owned multi-session impulse, pre-impulse trend, continuation-watch, ready and extended-risk states; counter-trend rebounds cannot become ready; confirmed day-two breakouts may create an immutable manual plan after live execution and results gates; day one, pullbacks and auto-buy remain blocked	`pipelines/trend_continuation.py`, `lib/trendContinuation.ts`
Armed Strong Swing	Earlier stage-2 breakout plans, blocking RS/regime gates, versioned 40-session time-of-day volume pace, structural stops and results blackout	`lib/armedSwing.ts`, `lib/analytics/armedSwing.ts`
News & AI Swing	Source-linked event overlay on existing technical candidates	`lib/news/`, `lib/newsSwing.ts`
Long-Term Candidates	Multi-year statement evidence and investor-inspired rankings	`lib/analytics/longTermStrategies.ts`
Probability	21-session return and risk distributions	`lib/probability-runtime.ts`, `lib/analytics/probability/`
Adaptive Allocation	Two-asset, cost-aware Universal Portfolio research and saved strategies	`lib/adaptiveAllocation.ts`, `lib/analytics/universalPortfolio.ts`
Forward tests	Recorded signal outcomes and comparison views	market forward-test page

The Stock Screener is at `/terminal/[market]/stocks`; `/terminal/[market]/screener` is Swing Candidates. AI dispatch supports Anthropic, OpenAI, Google and DeepSeek. Generated filters go through schema/filter validation and sanitization rather than becoming arbitrary SQL.

Armed Swing and execution plans

Armed Swing scans eligible active stocks, ranks relative strength and assesses a bounded shortlist of 250. Bases use completed sessions; current-session volume contributes to breakout pace against a market-specific, versioned curve containing at least 40 complete sessions. India benchmarks prefer NIFTY then NIFTYBEES; the US benchmark is SPY.

States are `BUY_ZONE`, `TRIGGERED_UNCONFIRMED`, `ARMED`, `UNVERIFIED`, `FORMING`, `EXTENDED`, `RISK_OFF` and `INVALIDATED`. Missing live quotes, bid/ask depth, circuit bands or verified results coverage blocks entry. India requires Breeze live execution data. Results within three trading sessions block new entries.

Server-held Armed plans freeze levels and gate evidence in immutable versions. Pending buy-stops require a same-session `ARMED` plan captured within 30 minutes. Direct Armed purchases require a matching `BUY_ZONE` plan and a purchase price within its trigger/limit band, subject to rounding tolerance.

Strong Swing and Momentum purchase validation likewise uses stored execution plans: actionable status, matching stock/market/strategy, current session, capture age at most 30 minutes, price at or above confirmed entry within tolerance and below target.

Trade Ledger and broker behavior

- Purchases, edit/delete, partial/final sales and sale correction.
- Direction-aware protection: long stops only tighten upward and short stops only tighten downward.
- Completed-close strategy exit lines and explicit planned first-target scale-outs for the five named strategy tags.
- Immutable purchase plans and current risk/news assessment.
- Pending buy-stop records with a default seven-calendar-day validity.
- Fill validation checks price band, remaining quantity and fill dates. Each confirmed partial fill creates its own purchased trade in the same database transaction.
- Unfilled pending orders have no position, capital employed or P&L. Cancelling the remainder preserves previous fills.
- Broker-reconciled acquisition value, proceeds and realized profit.
- Capital-employed ROI and annualized XIRR, with retained sale proceeds recycled before inferring new external capital.
- Breeze read-only holdings/positions/orders synchronization.
- Separately enabled automatic buying with spending limits and order intents for Strong Swing and saved Adaptive Allocation strategies; no automatic selling/rebalancing.

Recording a pending order or ledger fill does not submit a broker order.

Funds, credentials and automation

CAS import preserves ISIN/folio identities. AMFI bridges fund identifiers to AMC snapshots; Fund Mapping exposes composition and cross-fund overlap. Gmail discovery can process the latest CAS and matching AMC disclosures automatically when auto-import is enabled; ambiguous documents need review. Revoked grants set reconnect state, and scan errors/timestamps are surfaced.

Secrets are encrypted per user with AES-256-GCM. Settings responses expose configuration metadata and presence flags rather than decrypted credentials. SMTP, AI/news, Breeze and Gmail integrations retain distinct credential paths.

The Node wrapper coordinates market data, repair queues, fundamentals, macro, scans, forward tests, AMFI, news, mobile alerts, Gmail, results calendars and ML snapshots. Email defaults to 07:00 IST and uses saved per-user SMTP credentials plus India Strong Swing/Probability results. See email setup (docs/EMAIL_SETUP.md) for scheduling and retry limits.

Candidate scans evaluate every account whether or not it has an enabled Expo token. They persist status observations, refresh prioritized live-market targets and send a push only when an eligible enabled token exists. During an open India session, Momentum execution readiness requires both a fresh `BREEZE_LIVE` quote and a valid current-session Breeze OHLCV row with positive volume. Missing evidence leaves the candidate visible but execution-blocked.

Mobile

Expo SDK 57 / React Native 0.86.3 consumes versioned server APIs. Implemented features include secure login, India/US workspaces, server-ranked Strong Swing and News & AI, ledger mutations, candles, biometric re-entry and opt-in push.

Authenticated web pages also provide opt-in browser notifications for candidate-ready transitions and material open-trade actions. They use one server event per signature and market day, stable browser tags, and single-tab polling to prevent duplicate delivery while an InvestoGenie tab remains open. Candidate alerts are evaluated only during the relevant open market session, are discarded before delivery if the candidate is no longer ready, and link to the exact strategy section that produced them.

Ledger push text is generic. Ready-candidate push text includes ticker, readiness state and engine; Expo receives that text and the push token. Prices, quantities and broker details are excluded. See privacy (mobile/PRIVACY.md).

Local Prebuild/Gradle/Xcode is the documented distribution path. Existing package scripts still invoke EAS Build, and release validation reads EAS profile configuration; local builds require an explicitly supplied API URL. The source cannot establish which binary is installed.

ML and operational limits

The ML foundation provides schema, scheduled snapshots, deterministic Python features/labels and golden vectors. Training/calibration, promotion automation, monitoring enforcement and inference remain design contracts rather than complete runtime implementations. ML does not currently control candidate eligibility or broker execution.

Migrations extend through 0077. Migration 0077 adds versioned market-specific volume curves. The Linux release script discovers and applies migrations from 0033 onward after assuming an existing earlier schema, and re-executes the checked-out script once after pulling so migrations introduced by that release are not skipped. Provider quotas and continuing runtime health remain deployment-specific.

See AI review handoff (AI_REVIEW_HANDOFF.md), architecture (ARCHITECTURE.md), status (STATUS.md), deployment (DEPLOYMENT.md), ML methodology (docs/ml-methodology.md) and swing strategy rules and review (docs/swing-strategies.md).

Source document: STATUS.md

InvestoGenie Status

Code and production baseline reviewed 14 November 2026 through deployed revision `f33f256` on main.

This is a repository status reference. The production web/Breeze services, HTTPS login endpoint, schema through 0077, India results coverage and India Armed volume curve were checked at revision `f33f256`; broker account contents and installed mobile binaries were not inspected. The checks below are a dated baseline, not a guarantee of continuing provider or market-data health.

Implemented in the reviewed source

- Next.js 16.4.2 / React 19.2.4 web application with India/US terminal routes.
- Fundamental/price-action Stock Screener, natural-language filtering, saved screens and exports.
- Swing, Strong Swing, legacy/V2 Momentum Ignition, News & AI, Long-Term and Probability research surfaces.
- The Strong Swing page's four strategy engines are Python-authoritative for classifications, gates, rankings and trade levels. TypeScript loaders only assemble inputs and consume the Python batch results. The base Swing discovery scan and the separate Armed Swing screen have not yet been migrated.
- Armed Strong Swing at `/terminal/[market]/armed-swing`, separate from the existing Strong Swing gates/rankings.
- Immutable Armed and protected execution plans, with server-side ownership/session/status/price validation.
- Ledger pending buy-stops, partial fills, remaining-quantity tracking and cancellation.
- Broker-reconciled cash flows, retained-proceeds capital employed, ROI and XIRR.
- Cost-aware Adaptive Allocation backtests and separately enabled capped Breeze buy-only automation.
- CAS/AMFI fund mapping, AMC disclosures, fund overlap and optional Gmail auto-import.
- Gmail revoked-grant detection and connection error/reconnect state.
- Server-authoritative Expo mobile client with ledger mutations, News & AI, candles, biometrics and deduplicated alerts.
- Completed-session Momentum bases with separately verified live-session quote/OHLCV input, session-aware freshness and explicit execution blocking when evidence is absent.
- Account-wide candidate observation scans independent of push-token enrollment, persisted status transitions and two-minute Breeze live-target reconciliation.
- Python ML snapshot/feature/label foundation with golden-vector tests; no complete model training/inference runtime.

Latest schema additions

Migration	Purpose
0051	ML advisory foundation
0052	Corporate results calendar and sync state
0053	Immutable Armed Swing plans
0054	Pending buy-stop orders and fill audit
0055	Protected Strong Swing/Momentum execution plans

Migration	Purpose
0056	Authoritative results-calendar coverage window
0057	Gmail last-error timestamps and reconnect flag
0058	Persisted daily screener technical features and forward outcomes
0059	Candidate signal observation history
0060-0068	Trend continuation, browser-notification deduplication, signal ranking and retrospective evaluation
0069-0071	User roles, invitations and canonical system owner
0072-0076	Breeze history, quote order flow, intraday bars and broker order audit/actions
0077	Versioned market-specific intraday volume curves

The shared Linux release script discovers and applies migrations from 0033 onward in filename order. Fresh databases still need the full ordered schema. `scripts/verify-production-schema.mjs` checks required relations, including candidate observations, trend-continuation signals, browser-notification events, broker-order audits and market-volume curves. Browser notification signatures are unique per user and market day, and polling is serialized across open tabs. Trade-alert identity uses stable decision state rather than changing quote explanation text. After pulling, the release script re-executes the checked-out version once so a migration added in the same release is applied.

Authenticated web pages expose an opt-in browser-alert control. While the site is open in a foreground or background tab, it polls deduplicated server events and uses the browser Notification API for candidate-ready transitions and material ledger actions. Permission is requested only from an explicit user click; denied permission is never bypassed.

Trend Continuation v1 is an India-only, Python-calculated engine for persistent 3-5 session moves. It stores versioned impulse/watch/ready/extended states and appears on web and mobile Strong Swing surfaces. Day-one impulses and later pullbacks remain observational. A confirmed day-two breakout after 10:15 IST can create an immutable manual Buy & Track plan only when fresh Breeze depth, spread, circuit room, results-calendar, price-range and plan-freshness gates pass. It never feeds auto-buy.

Verified production baseline - 14 November 2026

- Revision `f33f256` was checked out; web and Breeze services were active and the public HTTPS `/login` returned HTTP 200.
- Migration 0077 was applied and the production schema verifier passed.
- Local release validation passed 573 TypeScript tests, 91 pipeline Python tests, 77 worker Python tests, typecheck, lint and the production build before promotion.
- Active NSE results-calendar verification covered 2,943 of 2,945 assets (99.93%) at the recorded check.
- The active India Armed curve is `in-median-2026-09-16-2026-11-13-40s`, built from 1,425 complete Breeze asset-sessions across 40 eligible sessions. A short special session was rejected by the completeness rule.
- US Armed remains fail-closed: Breeze is India-only and no equivalent US depth/intraday source is configured.
- The only local uncommitted file at deployment was machine-local `.claude/launch.json`; it was not included in the release.

Strategy review and changes - 13-14 November 2026

The formulas, gates and post-purchase handling of Swing Candidates, Strong Swing and Armed Strong Swing were reviewed at revision `30e6dd4` and the 14 findings were closed in source on 14 November 2026; see swing strategy rules and review (docs/swing-strategies.md) for each rule and change.

- Strong Swing is model `strong-swing-v4-breakout-bar`: breakout quality is judged on the bar that broke out, stocks holding a two-close breakout are assessed even when the base scan lists no setup, the trend needs the 50-day above the 200-day, a live price below the entry waits, and India entries need a verified results calendar with no results within three sessions. Plans and

captures from v3 are refused at purchase and by auto-buy.

- Legendary tags carry a liquidity floor, risk caps from the price paid and a breakout-day state with volume. Trades of the five named strategies bought from 14 November 2026 follow the strategy's own exit line in the ledger.
- Armed Swing requires the full moving-average stack, volume dry-up and a pivot at least three sessions old, and measures breakout volume against a time-of-day curve.
- Not yet observed over enough production decisions: the rate at which the new Strong Swing gates produce candidates. Five genuine scheduled v4 captures remain required and must not be reconstructed after the fact.
- India Armed has its required 40-session curve. US Armed remains fail-closed until a proper US execution-depth and intraday source is added.
- `npm run audit:strategy-readiness` is the pre-promotion gate: five scheduled v4 captures, at least one Execution Ready result per capture on average, at least 90% India results coverage, and an active 40-session volume curve per served market.

Current behavior that matters operationally

Armed entries fail closed when live execution evidence or results coverage is missing. India requires `BREEZE_LIVE`; calendar verification can use a complete same-day authoritative NSE window or usable dates under the fallback coverage policy. A successful request with no usable date is insufficient on its own.

Momentum discovery builds patterns from completed sessions and treats the current market session as a separate input. During an open India session, a fresh Breeze quote alone is insufficient: entry-ready classification also requires the matching valid Breeze OHLCV row and positive live volume. A missing or stale live input keeps the setup visible but execution-blocked. Freshness uses expected trading sessions, so weekends and exchange holidays do not create false gaps.

Pending orders are application records, not broker submissions. They carry no holdings/P&L until a user records a confirmed fill. Each partial fill becomes a separate purchased ledger trade while the remainder stays pending.

The scheduler wrapper is used by both `npm run dev` and `npm start`. Bare Next.js scripts do not run those schedules. Corporate results refresh is scheduled at 07:00 and 17:00 IST with startup work; the official NSE calendar is attempted before the Yahoo supplement.

The digest scheduler uses one global target time, default 07:00 IST, with catch-up and bounded retries. Although per-user `send_time` and `last_sent_at` are stored, the sender selects all enabled users without enforcing those fields. Retrying a partial batch can resend to users who already received it.

Mobile candidate pushes disclose ticker/readiness/engine to Expo, while ledger alerts remain generic. The documented local build path coexists with EAS Build scripts/configuration. App metadata is 0.1.1, Android version code 2, iOS build 2; package version remains 0.1.0.

Verification available in the repository

```
npm test
npm run lint
npm run build
npm run deploy:check:local
```

Mobile:

```
cd mobile
npm run typecheck
npm run validate:release
```

Focused tests exist for Armed Swing, plan validation, pending fills, execution plans, results-calendar ingestion, Gmail token errors and ML features/labels, alongside the wider analytics suite. Python tests are present in `pipelines/`, `workers/` and `scripts/`; they are not all invoked by `npm test`.

The Linux release runs production dependency audit, build and schema verification, then restarts services and checks `/login`. Tests and lint are separate pre-release steps. No application test run or provider-backed verification is claimed by this documentation review.

Remaining work and runtime checks

- Reconfirm the deployed revision, services and schema through the latest migration after each release.
- Inspect Data Health, quote/depth freshness, calendar coverage and provider failures.
- Review unresolved CAS/fund mappings and Gmail documents needing manual review.
- Confirm daily Breeze session validity and separately authorized buying limits.
- Validate a newly generated mobile binary on physical devices; source metadata is not install evidence.
- Implement and validate ML training, calibration, serving and monitoring before treating the design contract as an active model.
- Check email retry/deduplication behavior before relying on exactly-once delivery or individual send times.

See AI review handoff (AI_REVIEW_HANDOFF.md), capabilities (CAPABILITIES.md), architecture (ARCHITECTURE.md) and deployment (DEPLOYMENT.md).

Source document: DEPLOYMENT.md

InvestoGenie Deployment Targets

Source and production release path reviewed 14 November 2026 through deployed revision `f33f256`.

InvestoGenie supports local development/fallback and AWS production from the same Git revision.

Target	Command	Configuration	Database	Purpose
macOS local	`Launch InvestoGenie.command` or `npm run dev`	`.env.local`	Mac-local PostgreSQL	Development, manual research, local testing
macOS production rehearsal	`npm run build && npm start`	`.env.local`	Mac-local PostgreSQL	Verify the production build locally
macOS personal server	`npm run service:install`	`.env.local`	Mac-local PostgreSQL	Always-on private access through Tailscale
AWS Lightsail	systemd `investogenie.service`	`/etc/investogenie/investogenie.env`	VM-local PostgreSQL	Packaged production web service and scheduled ingestion
Oracle-compatible Ubuntu	systemd `investogenie.service`	`/etc/investogenie/investogenie.env`	VM-local PostgreSQL	Alternate deployment package

Detailed instructions:

- Local: `deploy/local/README.md` (deploy/local/README.md)
- AWS/Ubuntu production package: `deploy/oracle/README.md` (deploy/oracle/README.md)

Configuration isolation

The repository contains examples only. Real secrets remain outside Git:

- Local secrets: `.env.local`
- AWS/Ubuntu secrets: `/etc/investogenie/investogenie.env`

`CREDENTIAL_ENCRYPTION_KEY` is part of the data-encryption boundary. Preserve it when transferring a database that contains encrypted API or SMTP credentials. Local and Oracle may use different session/cron secrets, but restored encrypted credentials require the original credential-encryption key.

Promotion workflow

1. Develop and test locally.
2. Run `npm test`, `npm run lint`, `npm run build`, and

`npm run deploy:check:local`.

3. Commit and push the tested revision.
4. On AWS/Ubuntu, run `sudo deploy/oracle/deploy-release.sh`.
5. Run `npm run deploy:check:oracle` with the production environment loaded (the command name is

retained for compatibility).

6. Verify HTTPS, Data Health, scheduler logs, and a fresh PostgreSQL backup.

The release script installs locked build dependencies, rejects high-severity production dependency findings, builds the app, discovers and applies replay-safe migrations from `0033_breeze_credentials.sql` onward (currently through `0077_market_volume_curves.sql`), verifies required broker, news, mobile, allocation, ML, corporate-calendar, Armed/pending/execution-plan, screener and candidate-observation relations and calendar/Gmail columns, prunes development dependencies, and restarts both application services.

The launcher first updates the checkout, then re-executes the checked-out release script once under `INVESTOGENIE_DEPLOY_REEXEC=1`. This ensures a release that changes the deploy script or migration list uses the new version for that same deployment.

Application code is shared; environment files, databases, service management, and operational limits remain target-specific.

Migration and release limits

The release script assumes migrations 0001-0032 already exist. For a new database, apply all migration files in filename order before release; `npm run dev` does not provide a general migration runner.

The script performs `npm ci --include=dev`, a production dependency audit, build and pruning before Python dependency installation and migrations. It does not run `npm test` or lint; run those before promotion. Check that the selected Node runtime supports the flags used by the package scripts.

A successful source review cannot establish which revision or migrations are installed. Verify the actual host, database and services after deployment.

Source document: AGENTS.md

<!-- BEGIN:nextjs-agent-rules -->

This is NOT the Next.js you know

This version has breaking changes - APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in `node_modules/next/dist/docs/` (resolved from this file's directory; in monorepos the `next` package may not be visible from the repo root) before writing any code. Heed deprecation notices.

This block is written and re-added by `next dev` - verify at `node_modules/next/dist/server/lib/generate-agent-files.js`. Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

<!-- END:nextjs-agent-rules -->

InvestoGenie Agent Instructions

Start Here

InvestoGenie is an actively deployed personal market terminal for Indian and US markets.

Do not rediscover the repository from scratch.

Use these references according to the task:

- `ARCHITECTURE.md` - system architecture, repository boundaries, calculation boundaries, database domains, deployment, and verification.
- `CAPABILITIES.md` - currently implemented product functionality.
- `STATUS.md` - current repository/product state, recent changes, known issues, and remaining work.

For a normal focused coding task, do NOT read all three files completely.

Start with `ARCHITECTURE.md` when architectural context is required. Use `CAPABILITIES.md` only when existing product behavior/capability is relevant. Use `STATUS.md` only when current progress, recent fixes, deployment state, or known issues are relevant.

Prefer current source code over documentation when they disagree.

Context Efficiency

Keep exploration narrow.

- Read files directly related to the requested task first.
- Search for specific symbols/routes/functions instead of scanning directories broadly.
- Do not repeatedly read large documentation files during one task.
- Do not inspect `.claude/` unless explicitly requested.
- Do not inspect `.next/`.
- Do not broadly inspect `node_modules/`.

- For Next.js behavior, read only the relevant documentation under `node_modules/next/dist/docs/` as required by the Next.js rules above.
- Ignore historical AI context/conversation files unless explicitly requested.
- Use `git status` and targeted `git diff` to understand existing changes.

Expand exploration only when the current evidence is insufficient.

Repository Boundaries

Use `ARCHITECTURE.md` as the authoritative repository map rather than rediscovering directory structure.

Important boundaries include:

- `app/` - Next.js pages, routes, server actions
- `components/` - web presentation
- `lib/analytics/` - signal, ranking, risk, and strategy calculations
- `lib/news/` - news ingestion/evidence/classification
- `lib/cas/` - CAS parsing/import
- `lib/funds/` - fund identity/mapping
- `lib/breeze/` - ICICI Breeze integration and reconciliation
- `pipelines/` - Python data synchronization
- `workers/` - broker/market workers
- `scripts/` - scheduler, sync, migration, deployment utilities
- `db/migrations/` - ordered PostgreSQL migrations
- `mobile/` - isolated React Native/Expo Android/iOS client
- `deploy/` - deployment tooling

Architecture Rules

- Next.js/TypeScript serves the web application and server APIs.
- PostgreSQL is the system of record.
- Do not introduce Supabase.
- Trading calculations and ranking are server-authoritative.
- Mobile and web clients must consume server-produced trading results rather than independently reimplementing calculations.
- Preserve existing user scoping and parameterized-query patterns.
- Do not introduce a new framework, ORM, database, state-management system, or major dependency without explicit approval.

Protected Financial Logic

Treat the calculation/ranking layer as high-impact code.

This includes the protected modules documented in `ARCHITECTURE.md`, especially:

- `lib/analytics/`
- `lib/strongSwing.ts`
- `lib/swingTradeProjection.ts`
- `lib/swingTradeLedger.ts`
- `lib/newsSwing.ts`

Unless the user explicitly requests a strategy/calculation change:

- do not change formulas;
- do not change thresholds;
- do not change candidate ranking;
- do not change entry/target/stop/trailing-stop logic;
- do not change confirmation/execution gates;
- do not allow News & AI to create a technical setup;
- do not silently alter risk or probability assumptions.

For intentional financial-logic changes, inspect the relevant implementation and tests first and add/update tests.

Development Workflow

For each task:

1. Understand the requested outcome.
2. Check `git status`.
3. Inspect only relevant source and documentation.
4. For non-trivial work, form a short implementation plan.
5. Make the smallest coherent change.
6. Preserve existing architecture and conventions.
7. Add/update focused tests when behavior changes.
8. Run the cheapest relevant validation first.
9. Review the targeted diff.

10. Report what changed and what was actually validated.

Avoid unrelated refactoring.

Validation

Choose validation proportional to the change.

Prefer:

1. targeted tests/typecheck for the affected area;
2. broader tests when shared behavior changes;
3. full validation for substantial/release-critical work.

Repository-wide verification gates are documented in `ARCHITECTURE.md`.

Do not automatically run every expensive validation command for trivial changes.

Never claim that a test, lint, build, deployment, migration, or command succeeded unless it was actually executed successfully.

Database Safety

- Inspect relevant schema/migrations before database changes.
- Follow existing ordered migration conventions.
- Preserve `user_id` scoping for user-owned data.
- Use parameterized queries.
- Never destructively alter production data/schema without explicit approval.
- Never silently rewrite existing migrations that may already be deployed.

Secrets and Security

Never expose, print, commit, or hard-code:

- API keys
- passwords
- tokens
- SMTP credentials
- broker credentials
- encryption keys
- production environment secrets

Preserve the project's existing credential encryption and authentication boundaries.

Git Safety

Existing user work must be preserved.

- Never discard unrelated changes.
- Ignore unrelated `.claude` changes unless explicitly requested.
- Never use `git reset --hard`.
- Never use `git clean` to remove user files.
- Never force-push.
- Do not modify unrelated files merely to make the tree clean.
- Review relevant `git diff` before completion.
- Commit only when requested or when the user has explicitly delegated commits for the task.
- Push only when explicitly requested or clearly included in the user's requested workflow.

Deployment Safety

AWS Lightsail is the active production environment.

Do not deploy merely because a coding task is complete.

Deploy only when explicitly requested or clearly included in the requested task.

Before production deployment:

- verify the intended revision;
- run appropriate release validation;
- apply migrations deliberately;
- preserve production secrets/data;
- restart only the intended services;
- inspect deployment/service health after restart.

Do not provision or switch to Oracle Cloud unless explicitly requested.

Mobile Safety

`mobile/` is an isolated Android/iOS client.

Do not move server-authoritative trading calculations into the mobile application.

Use the existing versioned mobile API boundary.

Do not use Expo cloud build/update services unless explicitly requested. The chosen distribution path is local native Gradle/Xcode builds.

Communication

Keep responses concise and execution-focused.

At completion report:

- what changed;
- important files changed;
- validation actually run;
- remaining risks or follow-up.

Do not provide long explanations of unchanged repository areas.

Source document: README-screener.md

Stock Screener

Source reconciled 14 November 2026 through deployed revision `f33f256`.

The fundamental and price-action Stock Screener lives at `/terminal/in/stocks` and `/terminal/us/stocks`. The `/terminal/[market]/screener` route is the separate buy-side Swing Candidates workspace. There is no current top-level `/screener` page in the route tree.

Data flow

```
daily_ohlcv -> Python technical feature job -> screener_technical_daily
latest_quotes + latest_financials + screener_technical_daily
  -> lib/screener/snapshot.ts
  -> public.stock_snapshot
  -> lib/screener/service.ts / api/screener
  -> components/stock-screener/StockScreener.tsx
```

The snapshot is a persisted read model with one selected listing per symbol/market. Quote/price fields use latest quotes and historical bars; fundamentals come from normalized financial reports. Missing numeric values do not satisfy numeric comparisons.

`pipelines/screener_technical_features.py` computes SMA/EMA distance and pair relationships, RSI, ATR %, volume ratio, multi-session momentum and forward outcomes. The same frozen daily rows power current technical filters and historical tests. Backtests sample the last session of each week and report 5/20/60-session average, median and positive-return rate against the universe baseline. Current-only fundamentals are rejected rather than leaked into history.

`lib/screener/fields.ts` whitelists columns, labels and formats. The filter engine validates `{field, op, value}` clauses, combines them with AND and produces parameterized SQL. Presets define filter/sort combinations; sector-median value selection is resolved on the server.

Money displays use INR crore or USD millions according to row currency. Provider completeness varies; missing fields render as unavailable rather than invented zero values.

Field	Source
LTP, % change	`latest_quotes` (falls back to last OHLC close)
Volume, trade value, 52W high/low, gap %, intraday vol %	derived from `daily_ohlcv`
SMA/EMA relationships, RSI, ATR %, volume ratio, momentum	Python-derived from adjusted `daily_ohlcv`
Market cap, P/E, ROE, D/E, dividend yield, free cash flow, sector	yfinance `.info` via the fundamentals pipelines
ROCE, revenue/profit growth YoY	derived in the reports pipeline

Setup and refresh

Apply the ordered schema, including `0012_stock_screener.sql`. Seed membership and ingest the required data with the configured database environment:

```
node scripts/seed-universes.mjs
npm run sync:fundamentals
npm run sync:us-fundamentals
node scripts/refresh-screener.mjs IN
node scripts/refresh-screener.mjs US
```

Python package scripts expect `.venv/bin/python`. Latest quotes and OHLCV must also be populated for meaningful price-action screens.

`GET /api/cron/refresh-screener` supports a market selection and cron authorization. `vercel.json` registers `/15 3-10,13-21 * 1-5` UTC. The repository also contains the direct database refresh script and a macOS cron wrapper; their presence does not establish that a launchd job is installed on a particular host. Verify your actual refresh mechanism and avoid unintentionally running competing schedules.

Universes and saved screens

Static memberships are in `db/universes/` for Nifty 50/100/500, F&O and S&P 500. Re-seed after updating the source lists. ALL uses the tracked stock universe.

Saved screens are per-user rows in `public.saved_screens`, managed by `lib/screener-actions.ts`: save, reopen, rename and delete market/universe/filter/sort/column configuration. Signed-in users can also add stocks to their default watchlist.

Historical universe tests currently use today's constituent list and display a survivorship-bias warning. Point-in-time fundamentals and universe membership must be added before those dimensions can be claimed as bias-free backtests.

Natural-language filters

The signed-in action `parseScreenIntent()` resolves active provider configuration through `getActiveAIConfig()`. Anthropic, OpenAI, Google and DeepSeek dispatch is implemented in `lib/screener/nlQuery.ts`. Input is trimmed to 500 characters. Schema validation, filter validation and sanitization constrain output; the model does not supply arbitrary SQL. Configuration and provider errors surface to the caller.

This screener does not change Strong Swing, Armed Swing, Momentum or ML ranking rules and does not place broker orders.

Validation

`npm test` includes filter-engine and natural-language validation tests. Snapshot/database checks need a configured PostgreSQL instance; provider-backed AI calls require valid credentials. See architecture (ARCHITECTURE.md) and credentials (docs/SECURE_CREDENTIALS.md).

Source document: docs/swing-strategies.md

Swing, Strong Swing and Armed Strong Swing: rules and review

Rules as deployed 14 November 2026 at revision `f33f256` (Strong Swing model `strong-swing-v4-breakout-bar`). The review that led to them began at revision `30e6dd4`; production evidence below is explicitly dated.

The named-strategy thresholds are frozen as policy `legendary-v2-2026-11-14`; Armed rules are `armed-v3-2026-11-14`. Changing a threshold requires a new version and replay.

The in-app Help articles carry the same rules in reader form (`lib/help/articles.tsx`). When this file and the source disagree, the source wins.

1. Swing Candidates (`/terminal/[market]/screener`)

Sources: `lib/analytics/swingClassifier.ts`, `lib/analytics/legendaryStrategies.ts`, `lib/ingest/signals.ts` (hourly scan into `swing_signals`), `lib/screener.ts` (read-time levels and entry states).

Base classifier

Item	Rule
Breakout	close > highest high of the prior 20 bars, or close > upper Bollinger band (20, 2σ)
Squeeze	Bollinger bandwidth in the lowest 25% of its own history
Build-up	derivatives: price up and OI +5% over 5 bars; cash: price up over 5 bars and their average volume ≥ 1.2× the 20 bars before
Score	breakout 0.35 + squeeze 0.25 + build-up 0.30 + latest volume ≥ 1.5× 20-bar average 0.10
Entry	max(max(Donchian high, upper band), live price)
Stop / target	entry - 1.5 ATR(14); entry + 2 × risk
Trail	max(22-bar high, price) - 3 ATR
Entry state	`EXTENDED` when the live price is more than 1 ATR past the trigger

Legendary tags

A tag requires a 20-session average traded value of at least INR 1 crore (India) or $1M (US).

Strategy	Match (all required)	Entry	Stop	Trail	Risk cap from the price paid
Qullamaggie	40-session high ≥ 30% above the low of the prior 63 sessions; 10-40 sessions since; pullback ≤ 20%; later half of the consolidation holds a higher low in a range no wider; close > EMA10 > EMA20 with EMA20 rising; consolidation volume below the 20 sessions into the high; average daily range ≥ 4%; top 2% on 1/3/6-month return	consolidation high	max(consolidation low, entry - 1 ATR); on the breakout day, that day's low if higher	EMA10	-
Minervini	Trend Template 8/8 (criterion 8 = RS rating ≥ 70); last 2-4 contractions each shallower, final ≤ 15%, none deeper than 35%; close within ±5% of the pivot; volume since the pivot below the 50-session average; once above the pivot, that bar's volume above the 50-session average	pivot (latest swing-peak high)	max(final contraction low, pivot × 0.92)	SMA50	8%
Darvas	60-bar high that is also the 52-week high, ≥ 3 sessions old; first later low held by the next three sessions; that low unbroken; stop ≤ 15% below entry	one tick above the box top	one tick under the box bottom	the stop	15%

Strategy	Match (all required)	Entry	Stop	Trail	Risk cap from the price paid
PTJ	close above a rising SMA200 (vs 22 sessions ago), ≤ 15% above it, and the entry ≤ 15% above it; mirrored for shorts	10-bar high	SMA200	SMA200	15%
Simons	20-day z-score ≤ -2.5 with close above SMA200 (long) or ≥ +2.5 with close below it (short); one σ ≤ 15% of the close	close	close ∓ 1σ	stop; target = 20-day mean	15%

Tick: INR 0.05 in India, $0.01 in the US.

Breakout day (Qullamaggie, Darvas). `withBreakoutDay` re-reads the setup as it stood before the scan bar. If that bar traded through the entry, the tag stays for that bar with `triggered`, the bar's volume against the prior 50-session average, and `confirmed` when it closed at or above the entry on ≥ 1.5× volume.

Read-time levels and states. Entry = max(trigger, live price); the structural stop is tightened to the strategy's risk cap from that entry; target = the strategy's own, else entry + targetRR × (entry - stop). Each level carries an entry state: `BELOW_STOP`, `EXTENDED`, `BREAKOUT_CONFIRMED`, `BREAKOUT_UNCONFIRMED` or none.

After purchase

`lib/swingTradeProjection.ts` freezes the plan at the buy price. `calculateRecommendedGttStop` in `lib/swingTradeLedger.ts` is direction-sensitive: long protection may only rise and short protection may only fall. EMA/SMA strategy lines are completed-close exits, not intraday broker stops. Qullamaggie, Minervini, Darvas and PTJ plan a 50% first-target scale-out; Simons plans the full allocation.

Strategy	Trail (trades bought from 14 November 2026)
Default Swing, Strong Swing, Momentum, Armed, Trend Continuation	highest completed high after entry - `trailAtrMult` × ATR
Qullamaggie	completed close below the 10-day EMA
Minervini	completed close below SMA50
PTJ	completed close below SMA200 for long; above SMA200 for short
Darvas	one tick under the bottom of the newest box built since entry (`darvasTrailingBottom`)
Simons	none; the target is min(frozen target, current 20-day mean)

The moving-average line is evaluated only after a completed close. It does not replace the separate live catastrophic/broker stop. Trades opened before 14 November 2026 keep their prior follow-up contract.

2. Strong Swing (`/terminal/[market]/strong-swing`)

Sources: `pipelines/strategy_math/strong_swing.py` (authoritative), `lib/analytics/strongSwing.ts` (test oracle), `lib/strongSwing.ts` (inputs, snapshot), `lib/resultsCalendar.ts`, `lib/swingExecutionPlans.ts` (purchase validation), `workers/breeze_auto_buy.py`, `pipelines/strong_swing_parity.py`.

Universe: every long base-classifier setup, plus every stock whose last two completed closes are above the highest high of the 20 sessions before them, whatever the base scan calls it. NSE only for India. Bars are completed sessions; today's bar is excluded while the market is open.

```
breakout level = highest high of the 20 completed sessions ending two bars back
entry          = breakout level + 0.25 × ATR(14)
stop           = entry - stopAtrMult × ATR        (default 1.5)
target         = entry + targetRR × (entry - stop) (default 2)
breakout bar   = the latest bar if it broke out, else the previous bar if it did
```

Blocking technical gates: structural breakout on either of the last two bars; close > SMA50 and SMA200, SMA20 > SMA50 > SMA200, SMA50 rising; 20-session return above the benchmark's; on the breakout bar: close ≥ level + 0.25 ATR, close in the top 30% of its range, volume ≥ 1.5× the 20 sessions before it; both latest closes ≥ entry; participation (5-session volume ≥ 1.2× the 20-session average with price up, or OI +5%).

Blocking execution gates: fresh data; price ≥ INR 20 / $2; 20-session traded value ≥ INR 1 crore / $1M; live price ≥ entry × 0.999 and ≤ entry + 0.5 ATR; 5-session return ≤ 30%; 10-session ≤ 50%; ≤ 15% above SMA20; ATR ≤ 5%; stop ≤ 7%; fewer than two circuit-like sessions in 20; benchmark not down 4% in two sessions; India: verified results calendar with no results within three sessions; India live session: Breeze quote ≤ 5 min, spread ≤ 0.75%, ≥ 3% above the lower circuit when the band is known. Advisory: market regime (benchmark above a rising SMA50), and the results check in the US, where the calendar has no coverage.

Status order: `INVALIDATED` (latest close < level - 1 ATR) → `RISK_OFF` (hard execution gate, results included) → `WAIT_FOR_ENTRY` (technically confirmed, outside the buy zone or above the SMA20 limit) → `EXECUTION_READY` → `WATCHLIST`.

Purchase validation refuses a fill below entry × 0.999, above entry + 0.5 ATR, or at/above the target. Auto-buy additionally requires the market-regime gate to pass and the current model version.

3. Armed Strong Swing (`/terminal/[market]/armed-swing`)

Sources: `lib/analytics/armedSwing.ts`, `lib/armedSwing.ts`, `lib/armedSwingPlans.ts`.

Universe: active stocks with ≥ 60 bars, price and liquidity floors, price between 90% and 105% of the 20-session high; the 250 highest by relative strength are assessed.

```
pivot   = highest high of the last 20 completed sessions
trigger = pivot + 0.1 ATR;  limit = min(trigger + 0.25 ATR, pivot × 1.03)
stop    = min(low of the last 5 sessions - 0.1 ATR, trigger - 1 ATR)
target  = trigger + targetRR × (trigger - stop);  trail = max(stop, trigger - 3 ATR)
volume pace = today's volume ÷ (20-session average × expectedVolumeShare(session progress))
```

`expectedVolumeShare` uses the active curve for that market. Curves are versioned median cumulative-volume profiles built from at least 40 completed sessions; a triggered setup cannot confirm when its market has no valid curve.

Blocking technical gates: stage-2 trend (close > SMA50, SMA150 and SMA200; SMA50 > SMA150 > SMA200; SMA200 rising over 20 sessions; within 25% of the 52-week high; ≥ 30% above the low); RS percentile ≥ 80; RS line within 2% of its six-month high; base depth ≤ 18%; second-half range ≤ 75% of the first half or bandwidth in the lowest 25%; pivot high set ≥ 3 completed sessions ago; five-session volume ≤ 0.85× the 50-session average; within 1.5 ATR and 6% of the trigger; once triggered, volume pace ≥ 1.5×.

Blocking execution gates: fresh data, price, liquidity, ATR ≤ 5%, stop ≤ 8%, circuit behaviour, market shock, benchmark above a rising SMA50, verified results calendar with no results within three sessions, live quote, spread ≤ 0.75%, both circuit bands with ≥ 3% lower room and the limit inside the upper band.

4. Review findings and how each was closed

The review found no arithmetic error. The findings were places where the rules, the funnel or the follow-up after purchase departed from the method a strategy is named for.

#	Finding (13 November 2026)	Change (14 November 2026)
1	Strong Swing tested breakout-day volume, close and clearance on the hold day. No `EXECUTION_READY` in 14 sessions of scheduled captures.	Those three gates are measured on the bar that broke out.
2	Strong Swing assessed only current base setups; 43 of 90 NSE stocks holding a two-close breakout were `NO_SETUP` and never checked.	Stocks holding two closes above the breakout level join the universe.
3	The ledger trailed every strategy by ATR.	Qullamaggie, Minervini, PTJ and Darvas trades follow their own line; Simons follows the moving mean. New trades only.
4	Qullamaggie and Darvas tags vanished on the breakout, with no volume test.	The breakout day keeps the tag with a triggered state, its volume ratio and a confirmed flag; shown on the Swing page.

#	Finding (13 November 2026)	Change (14 November 2026)
5	Darvas and PTJ stops had no width limit (average 25% and 11%).	Darvas boxes over 15% are not tagged; PTJ's 15% is measured from the entry; every strategy stop is capped from the price paid.
6	Strong Swing had no results-date gate.	Results blackout gate, blocking in India.
7	Strong Swing stayed ready below its entry; purchases had no upper bound short of the target.	Below the entry is `WAIT_FOR_ENTRY`; purchases above entry + 0.5 ATR are refused.
8	Strong Swing trend did not require SMA50 > SMA200; market regime blocked nothing.	SMA50 > SMA200 required. Regime stays advisory for manual entries; auto-buy requires it.
9	Minervini: 8% cap from the pivot, no breakout volume, unbounded first contraction.	Cap from the price paid; above-average volume once through the pivot; contractions ≤ 35%.
10	Armed: looser trend, advisory dry-up, no pivot age.	Full 50 > 150 > 200 stack with close above the 50-day; dry-up required; pivot ≥ 3 sessions old.
11	Armed volume pace was linear in time.	Pace against the time-of-day volume curve.
12	Simons: no trend filter, fixed target, very wide σ rows.	200-day trend filter, σ ≤ 15% of price, liquidity floor, target follows the mean after purchase.
13	Base classifier: no extended state; a price below a strategy stop not flagged.	`EXTENDED` and `BELOW_STOP` entry states at read time.
14	Darvas used a 0.01 offset regardless of tick.	Entry and stop on the exchange tick.

Left as designed

- Base classifier breakout and filters. A close above the upper Bollinger band still counts as a breakout, and the base list still has no trend, liquidity or market filter. It is the discovery feed for Strong Swing, whose own gates require clearance of the 20-session high, a trend and liquidity; changing the feed would alter every downstream engine.
- Completed-bar discovery with a live overlay. The hourly scan calculates patterns only from completed daily bars. A fresh live quote may mark a frozen trigger as crossed, but that state remains unconfirmed until the completed bar supplies its close and volume.
- Armed leadership and regime are blocking. Weak relative strength or a weak market regime prevents an actionable Armed status.
- Partial profit-taking is explicit. Qullamaggie, Minervini, Darvas and PTJ plans schedule 50% at the first target; Simons exits the full mean-reversion allocation. Fills remain user-confirmed ledger events.
- Close-only exits stay close-only. EMA/SMA strategy lines are evaluated against completed closes and do not replace the live catastrophic broker stop.

Production evidence and remaining observation

Revision `f33f256` is deployed. India results-calendar verification covered 2,943 of 2,945 active NSE assets (99.93%) at the 14 November check. The active India Armed curve `in-median-2026-09-16-2026-11-13-40s` contains 1,425 complete Breeze asset-sessions across 40 eligible sessions; the short special session was excluded by the completeness rule. US Armed remains fail-closed because Breeze does not supply US equities and no equivalent US depth/intraday source is configured. Five genuine scheduled Strong Swing v4 captures are still required to establish post-deploy candidate frequency; they must not be reconstructed after the fact.

Source document: docs/ml-methodology.md

InvestoGenie ML Methodology - Production Contract v1

Contract reconciled 14 November 2026 through deployed revision `f33f256`. The Python snapshot/feature/label foundation is present; no validated model serving assignment was established.

Implemented versus specified

Implemented source includes migration `0051_ml_advisory_foundation.sql`, `scripts/ml-snapshot-scheduler.mjs`, `scripts/capture-ml-snapshots.mjs`, Python `collect_snapshots.py`, `features.py`, `labeler.py`, golden vectors and feature/label tests.

The model lifecycle, promotion gates, walk-forward training, calibration, serving and drift response below are design requirements. Complete training/calibration/inference, promotion and monitoring automation are not present in the repository. Tables and policy contracts alone do not implement those services. A current serving assignment requires database inspection.

Migration 0051 is the ML schema boundary; the application schema extends through `0077_market_volume_curves.sql`. Later Armed/pending/execution-plan, screener-feature, candidate-observation and market-volume-curve workflows do not extend ML inclusion statuses or authorize ML broker buying.

Purpose and boundary

The intended ML layer estimates the probability that an already eligible Strong Swing candidate reaches its frozen target before its frozen stop. It is an advisory rank only. It cannot create a candidate, bypass a technical or execution gate, send an alert while in shadow, or place a broker order.

Python is the only calculation runtime for feature construction, labeling, training, calibration, evaluation and inference. Next.js schedules Python jobs and reads persisted results; web and mobile clients only render them. A calculation must not be independently reimplemented in TypeScript.

Frozen production policies

Contract	Version/value
Feature version	`strong-swing-features-v2`
Label version	`strong-swing-first-touch-v1`
Entry fill policy	`strong-swing-entry-v1`
Trailing policy	`strong-swing-trailing-v1`
Live-ledger trailing policy	`strong-swing-ledger-trailing-v2-post-entry-next-bar`
Inclusion policy	`strong-swing-entry-eligible-v1`
India calendar	`NSE-calendar-v1`
Canonical India decision time	18:45 Asia/Kolkata, 30-minute grace
Included candidate statuses	`EXECUTION_READY`, `WAIT_FOR_ENTRY`
Maximum entry extension	0.50 ATR
Break-even activation	0.75 initial-risk multiple
Stop update timing	next expected trading session only
Entry slippage	10 bps

Contract	Version/value
Exit slippage	10 bps
Round-trip charges	10 bps

Costs are conservative modeling assumptions, not broker invoices. Changing a value above requires a new policy or label version and complete re-evaluation.

Canonical data

The scheduled collector writes at most one snapshot per `(market, market_date, decision_time, asset_id, source, feature_version)`. Page visits and ad-hoc evaluations are `interactive`; only `scheduled` rows are eligible for training or evaluation. The snapshot records its data cutoff, calendar version and scheduler run.

For India, `scripts/ml-snapshot-scheduler.mjs` waits for both the 18:45 decision point and a successful `eod-in` record for the same exchange session. It then freezes the candidate set and invokes `pipelines/ml/collect_snapshots.py`. Reruns reuse the canonical candidate key and never duplicate the immutable Python feature row.

Universe membership, liquidity and market-cap buckets are point-in-time data. Present-day `is_active` state and broker mappings must not determine historical eligibility. OHLCV uses the same split/bonus adjustment source as the portfolio engine. Delisted and renamed assets retain their immutable asset IDs. The v1 collector records a point-in-time liquidity bucket and explicitly marks market-cap as `UNKNOWN`; model training remains blocked until historical market-cap reconstruction is available or the registered feature contract excludes that field.

Label semantics

Entry scanning starts on the first expected session after the decision. A crossing fills at the frozen confirmation entry. A gap at or above entry fills at the opening price only when it is no more than 0.50 ATR beyond entry; a more extended gap is skipped. The entry session counts as session one.

For every post-entry session, the labeler first applies the stop frozen from the preceding session. An opening gap below the stop exits at open; an opening gap above the target exits at open. Otherwise it evaluates the bar's low against the effective stop and high against the target. If both are touched, the raw outcome is `SAME_BAR_AMBIGUOUS` and the binary training target is zero. The current high may raise the stop only for the next session. Stops never decrease.

Outcomes are `TARGET_FIRST`, `STOP_FIRST`, `SAME_BAR_AMBIGUOUS`, `WINDOW_EXPIRED`, `NOT_ENTERED`, or `INVALID_DATA`. Holidays come from the recorded calendar and are not missing sessions. A missing expected session or invalid OHLC value is `INVALID_DATA`. MFE and MAE stop at the outcome bar. Returns are net of the frozen modeled costs.

The implementation is `pipelines/ml/labeler.py` (../pipelines/ml/labeler.py). Decimal arithmetic and canonical JSON outputs make repeated runs deterministic. `pipelines/ml/golden_vectors.json` (../pipelines/ml/golden_vectors.json) is the executable policy contract.

Model lifecycle and serving contract

Models move through `CANDIDATE`, `SHADOW`, `VALIDATED`, `SUSPENDED`, and `RETIRED`. Lifecycle and serving are separate: `ml_serving_assignments` holds at most one served model per market, and rollback is an atomic pointer swap. Every transition or assignment requires a recorded promotion decision. Model artifacts are immutable and checksum verified before loading.

The first model is pooled across India candidates; ticker identity is prohibited. Permitted categoricals are point-in-time liquidity, market-cap and regime buckets. Training uses scheduled snapshots matching the declared inclusion, feature, label, trailing, entry and calendar versions.

Walk-forward folds are `training → purge → calibration → purge/embargo → test`. Both gaps are at least the maximum holding window. Test decision dates do not overlap. Sigmoid calibration is the default. Test data never fits either model or calibrator.

Registered promotion gates

These initial thresholds must be inserted before the final test is inspected:

- At least 1,000 independent test decision-date clusters and 200 positive clusters.
- At least five accepted folds; each accepted fold has 100 clusters and 20 positives.
- Top-decile precision improvement over baseline at least 5 percentage points, with a 95% bootstrap confidence-interval lower bound above zero.
- Top-decile net expectancy after frozen costs greater than zero, with a 95% bootstrap confidence-interval lower bound at least zero.
- Brier score at most 0.22 and 10-bin ECE at most 0.08.
- Adverse-regime precision deterioration versus its unranked baseline no worse than 5 percentage points.
- Fold top-decile precision-improvement standard deviation at most 10 percentage points; no accepted fold may be negative by more than 5 percentage points.

Insufficient folds are reported, never silently discarded. Threshold changes after test inspection invalidate the run.

Monitoring and failure-behavior contract

Production monitoring covers calibration, feature drift, invalid-label rate, ranked-versus-baseline performance and source freshness. Confirmed drift atomically removes the serving assignment, records the observation, notifies the operator and falls back to unranked Strong Swing. Data-pipeline failures alert separately. An unavailable or invalid model never blocks Strong Swing.

No model from this lifecycle may drive screen-based or broker buying. Any future execution use requires a separate specification and promotion process.

Current implementation state

The schema, scheduled canonical snapshot collector, deterministic Python feature builder/labeler and shared golden vectors are the production foundation. Interactive snapshots remain excluded from training. Training eligibility is limited to `EXECUTION_READY` and `WAIT_FOR_ENTRY`; ticker identity and raw price are excluded from the feature vector. Until a model passes the registered gates and receives an explicit serving assignment, Strong Swing remains unranked by ML and all alerts and Breeze buying continue to use their existing non-ML controls.

Verification

The Python feature and label regression tests are `pipelines/test_ml_features.py` and `pipelines/test_ml_labeler.py`; they consume the feature/label code and golden vectors. They require a Python environment and are separate from `npm test`. Scheduled collection needs PostgreSQL and the matching schema. This documentation review did not execute database collection or evaluate a trained model.

Source document: docs/EMAIL_SETUP.md

Email Digest Setup

Source reconciled 14 November 2026 through deployed revision `f33f256`; known scheduling limits remain listed in `STATUS.md`.

The daily digest uses the same server sources as India Strong Swing and Probability. Each enabled section includes up to five rows. Email delivery does not place orders.

Configuration

1. Apply the database migrations, including 0020 email preferences, 0021 credentials and later credential schema updates.
2. Set `CREDENTIAL_ENCRYPTION_KEY`, `CRON_SECRET` and the correct `NEXT_PUBLIC_APP_URL` in the environment.
3. Sign in, open Settings and save the user's SMTP host, port, username and password under secured credentials.
4. Enable Email digest and select its content.

`lib/email-actions.ts:sendEmailDigest()` loads SMTP from that user's `user_credentials` row and decrypts the stored password. It does not call the environment-fallback SMTP resolver. Missing saved credentials produces `SMTP credentials not configured for user`.

The transport uses TLS mode according to port configuration; use the settings required by your SMTP service. SMTP authentication is separate from the read-only Gmail disclosure OAuth integration.

Scheduling

Both `npm run dev` and `npm start` invoke `scripts/run-with-nse-sync.mjs`, including its daily digest scheduler. Keep the application running to receive scheduled work. Bare `dev:next`/`start:next` commands skip the wrapper.

Variable	Default	Meaning
`EMAIL_DIGEST_HOUR_IST`	7	Global send hour in IST
`EMAIL_DIGEST_MINUTE_IST`	0	Global send minute
`EMAIL_DIGEST_CRON_DISABLED`	unset	Set 1 to disable wrapper delivery
`EMAIL_DIGEST_MAX_ATTEMPTS`	5	Attempt budget per IST day
`EMAIL_DIGEST_RETRY_MINUTES`	5	Retry delay

The scheduler checks every minute, supports catch-up after the target and seeds its last successful day from database logs. A partial/error response can trigger another bounded attempt.

The UI stores a per-user `send_time`, but the current scheduler does not use it. The enabled-user query also does not filter by `last_sent_at`. Successful sends update that timestamp, but it is not an exactly-once guard: a retry after a partial batch may resend to users whose earlier send succeeded.

Manual and external invocation

`GET /api/cron/send-email-digest` accepts the configured `Authorization: Bearer` header. Configure a real nonempty `CRON_SECRET`; do not rely on missing-secret behavior.

The following sends real email to all enabled users; export the configured secret in the shell first:

```
curl http://localhost:3000/api/cron/send-email-digest \
  -H "Authorization: Bearer $CRON_SECRET"
```

Responses include `status` (`success`, `partial` or failure), `sent`, `errors`, elapsed `duration` and error details. A partial batch is reported in the JSON body, so inspect it even when HTTP succeeds.

If running without the wrapper, use one external scheduler. 07:00 IST equals 01:30 UTC; a five-field UTC cron expression is `30 1 *`. The committed `vercel.json` does not currently contain the digest endpoint. Disable wrapper delivery if intentionally moving scheduling elsewhere.

Monitoring and troubleshooting

```
select *
from public.cron_logs
where job = 'send-email-digest'
order by created_at desc
limit 10;
```

- Missing SMTP: save complete credentials for each enabled user.
- Decryption failure: restore the original encryption key or re-enter the affected secret.
- Unauthorized request: ensure the bearer matches the configured cron secret.
- Missing scheduled send: inspect wrapper uptime, disable flags, global IST target, retry budget and logs.
- Repeated email: check manual callers, multiple schedulers and partial-batch retries.
- Empty candidates: inspect the India Strong Swing and Probability data sources; there is no separate documented freshness gate in the mail sender.

The route processes users independently and logs a batch summary; one user's failure does not prevent attempts for the others.

See quick start (QUICK_START_CREDENTIALS.md) and credential storage (SECURE_CREDENTIALS.md).

Source document: docs/QUICK_START_CREDENTIALS.md

Quick Start: Credentials and Email Digest

Source reconciled 14 November 2026 through deployed revision `f33f256`; credential behavior was not changed by the strategy release.

1. Prepare the application

Follow local setup (../deploy/local/README.md) or deployment (../DEPLOYMENT.md), including the ordered database migrations. Copy `.env.example` to `.env.local` for local use.

Generate separate random secrets:

```
openssl rand -hex 32
```

Copy generated values into named assignments; do not append an unlabeled random line to the environment file.

```
CREDENTIAL_ENCRYPTION_KEY=YOUR_GENERATED_VALUE
CRON_SECRET=A_DIFFERENT_GENERATED_VALUE
SESSION_SECRET=ANOTHER_GENERATED_VALUE
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Keep exactly one nonempty app URL assignment. Preserve the original credential key when restoring an encrypted database.

2. Save SMTP credentials

Start `npm run dev`, sign in and open `/settings`. In secured credentials, enter your SMTP host, port, username and password and save. The digest sender specifically requires the signed-in user's stored SMTP credentials; SMTP environment variables alone do not configure it.

3. Enable the digest

Enable Email digest and select Strong Swing and/or Probability content. Current digest data is for India and includes up to five rows per selected section.

The wrapper schedules all enabled users at a global time: 07:00 IST by default. The per-user time field is saved but does not control scheduling in the current implementation.

4. Optional AI/news setup

Settings supports an active Anthropic, OpenAI, Google or DeepSeek provider/model/key. News supports Marketaux, GNews, NewsAPI and Alpha Vantage. AI/news keys are independent of SMTP. Saved secrets are encrypted; the UI receives presence flags.

5. Verify

`npm run dev` and `npm start` already run the scheduler. No external cron is required for that setup. A host with only Next.js running needs a separate scheduler.

An operator can trigger an actual send to all enabled users with the configured secret exported in the shell:

```
curl http://localhost:3000/api/cron/send-email-digest \
  -H "Authorization: Bearer $CRON_SECRET"
```

Inspect `sent`, `errors` and `errorList`, then check `cron_logs` for `send-email-digest`. Repeated manual calls and partial retries can resend to successful recipients; the sender does not enforce `last_sent_at` deduplication.

See email setup (EMAIL_SETUP.md) for scheduling controls and credential security (SECURE_CREDENTIALS.md) for storage and recovery.

Source document: docs/SECURE_CREDENTIALS.md

Credential Storage

Source reconciled 14 November 2026 through deployed revision `f33f256`; credential behavior was not changed by the strategy release.

Implementation

`lib/crypto/credentials.ts` uses AES-256-GCM. A 32-byte key is derived from `CREDENTIAL_ENCRYPTION_KEY` using scrypt and the fixed application salt `investogenie-credential-salt`. Each encrypted value uses a random 16-byte IV and an authentication tag.

Stored format:

```
iv:authTag:ciphertext
```

All three components are hexadecimal. The master key comes from the environment, not the database. This encrypts selected secret fields; it does not encrypt the entire database or backups.

Credential domains

Data	Storage/resolution
SMTP password and host/port/user metadata	`user_credentials`; password encrypted
Active AI provider/model/key	Provider/model metadata plus encrypted key
News provider keys	Legacy credential fields and multi-provider `user_news_providers`
Breeze API key, secret and session token	Encrypted user credential fields
Gmail OAuth tokens and optional CAS password	Gmail connection domain, encrypted secrets
Mobile device session	Raw token in SecureStore; only SHA-256 hash on server

The baseline credential schema is migration 0021; later migrations extend AI, news and Breeze. Gmail health fields are added by 0057. Apply the full ordered schema for a fresh database; startup is not a general migration runner.

Setup and recovery

Generate a high-entropy master value:

```
openssl rand -hex 32
```

Save it as `CREDENTIAL_ENCRYPTION_KEY=...` in local `.env.local` or the deployment's protected environment file. Generate session/cron secrets independently.

Preserve the original master key alongside recoverable backups using a separate protected storage location. Restoring encrypted records under another key will not decrypt them. Changing the variable alone is not credential rotation: decrypt/re-encrypt under controlled migration or re-enter secrets. If the original key is lost, existing encrypted secrets cannot be recovered.

Settings and server access

`lib/credentials-actions.ts` requires authentication for user-facing updates and scopes rows to the user. `getUserCredentials()` returns client-safe metadata and flags such as `smtpPasswordSet` and `aiApiKeySet`. It does not return decrypted passwords or keys.

`updateCredentials()` encrypts submitted secret values before writing. Clearing is explicit through `clearCredential()`; blank update fields preserve existing encrypted values.

Server-only resolvers such as `getActiveAIConfig()`, `getSystemAIConfig()`, news resolvers and `getSMTPConfig()` may return plaintext secrets for provider calls. Never pass these results to client props, logs or API responses.

```
import { encryptCredential, decryptCredential } from "@/lib/crypto/credentials";

// Server-only operations; do not log either value.
const stored = encryptCredential(secret);
const restored = decryptCredential(stored);
```

There is no current `getAIProviderKey()` API. The natural-language screener resolves the chosen provider/model/key through `getActiveAIConfig()` and dispatches through `lib/screener/nlQuery.ts`.

Fallbacks are caller-specific

Interactive AI first uses the user's saved configuration; system AI has its own environment/default-user resolution. Multi-provider news resolution differs from legacy single-provider fallback. Read the relevant resolver before assuming an environment key is used.

`getSMTPConfig()` supports database/environment resolution for its callers. The daily digest does not use that helper: `sendEmailDigest(userId)` requires that user's stored SMTP host/user/encrypted password. Environment `SMTP_*` values alone do not enable digest delivery.

Gmail refresh errors can set `needs_reconnect`; Fund Mapping exposes the last scan error/timestamp. A new OAuth consent clears reconnect/error state on successful reconnection. Repeated failures require inspecting the connector's reported error, not changing the encryption key.

Troubleshooting

- Missing master-key error: configure the same key in every process that reads encrypted records.
- Invalid encrypted format: verify the stored three-part value and restore a valid backup or re-enter it.
- Authentication/decryption failure after restore: verify the original master key before replacing records.
- Credentials appear unset: confirm the user and saved presence flags; do not inspect secrets in logs.
- Provider authentication failure with valid storage: verify the provider key/session separately.

Access control, HTTPS, host security and protected backups remain separate controls. This storage implementation alone is not evidence of regulatory certification.

See email setup (EMAIL_SETUP.md), quick start (QUICK_START_CREDENTIALS.md) and deployment (../DEPLOYMENT.md).

Source document: deploy/local/README.md

Local macOS deployment

Source reconciled 14 November 2026 through deployed revision `f33f256`; this guide covers the local macOS package.

Local mode is the development and data-workstation deployment. It uses the Mac's PostgreSQL instance and `.env.local`; it does not read Oracle production secrets or connect to the cloud database.

First setup

Install Node.js 20+, Python 3, and PostgreSQL. Then:

```
cd /path/to/investogenie
cp .env.example .env.local
openssl rand -hex 32
```

Put separate generated values into `SESSION_SECRET`, `CRON_SECRET`, and `CREDENTIAL_ENCRYPTION_KEY`. Preserve the credential key when restoring an existing database containing encrypted credentials.

Create/start PostgreSQL and install dependencies:

```
brew services start postgresql@16
createdb investogenie 2>/dev/null || true
npm ci
python3 -m venv .venv
.venv/bin/pip install -r pipelines/requirements.txt
.venv/bin/pip install -r workers/requirements.txt
# Fresh empty database only: export DATABASE_URL from your configured environment.
for migration in db/migrations/*.sql; do
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$migration" || break
done
npm run deploy:check:local
```

Confirm every migration completed before starting. The schema extends through `0059_candidate_signal_observations.sql`. Existing databases require applying only their missing migrations in order; inspect their schema and take a backup first. `npm run dev` does not automatically apply the complete migration chain.

For an existing machine, use the one-click `Launch InvestoGenie.command` file. It starts PostgreSQL when possible, prepares dependencies, launches the app and opens `http://localhost:3000`.

Always-on personal service

Install InvestoGenie as a macOS login service when this Mac is the personal server:

```
npm run service:install
```

The installer builds the production application and registers `com.investogenie.app` with `launchd`. It starts after login, restarts after an unexpected failure, binds the application to `127.0.0.1:3000`, and keeps all existing quote, history, fundamentals, AMFI and email schedules in the main application process. PostgreSQL remains managed by Homebrew.

Useful commands:

```
npm run service:logs
launchctl print gui/$(id -u)/com.investogenie.app
npm run service:uninstall
```

Run `npm run service:install` again after pulling application changes. This rebuilds the production bundle and restarts the service. Logs are written to the ignored `logs/` directory.

Gmail mutual-fund disclosures

Fund Mapping can discover monthly AMC portfolio attachments in a connected Gmail account. The connector requests only Gmail read access; it stores encrypted OAuth tokens and message metadata. Discovery and enabled auto-import can fetch attachment bytes during processing; attachment bytes and email bodies are not retained as document storage.

1. In Google Cloud Console, enable the Gmail API for a project.
2. Configure the OAuth consent screen as External/Testing and add the Gmail

account as a test user.

3. Create an OAuth 2.0 Web application client.
4. Add `http://localhost:3000/api/gmail/callback` and the Tailscale HTTPS origin

followed by `/api/gmail/callback` as authorized redirect URIs.

5. Put the client ID and secret in `.env.local`:

```
GOOGLE_GMAIL_CLIENT_ID=...
GOOGLE_GMAIL_CLIENT_SECRET=...
GMAIL_DISCLOSURE_SYNC_INTERVAL_HOURS=24
```

Restart the service, open Portfolio > Fund Mapping, and choose Connect Gmail. The startup wrapper scans connected inboxes once at startup and every 24 hours. With auto-import enabled, suitable latest CAS statements and unambiguous AMC fund/month matches can be imported automatically. Ambiguous documents remain in Needs review. Configure the encrypted CAS PDF password when required. Revoked grants show a reconnect requirement, and scan failures expose their last error/timestamp; reconnect Gmail when prompted.

Google classifies `gmail.readonly` as a restricted scope. Personal use in OAuth testing mode is appropriate; a public multi-user release requires Google's OAuth verification process and may require an additional security assessment.

Private access with Tailscale

For personal remote access, install Tailscale on this Mac and the devices that should open InvestoGenie. Double-click `Install Tailscale.command` and approve the macOS administrator/network-extension prompts. Once both devices are signed in to the same tailnet, enable Tailscale Serve and open the private HTTPS URL it reports:

```
https://<mac-name>.<tailnet-name>.ts.net
```

No router port forwarding or public PostgreSQL port is required. Both the app backend and PostgreSQL remain bound locally; Tailscale Serve is the private HTTPS entry point. Tailscale does not provide a fixed public outbound IP for broker API allowlisting.

The service uses `caffeinate` to prevent idle system sleep while the Mac is connected to AC power. Closing a laptop lid, logging out, or losing power can still stop scheduled updates.

Local password recovery

Double-click `Reset InvestoGenie Password.command` on the host Mac to reset an existing account. The helper requires physical access to the Mac, hides the new password while it is entered, and updates only the password hash. It does not delete or move portfolio data. The login reset option is additionally available when `PASSWORD_RESET_KEY` is configured; it is gated by that key. The host helper remains available without that configuration.

Local run modes

Development with hot reload and all background schedules:

```
npm run dev
```

Production-mode rehearsal on the Mac:

```
npm run build
npm start
```

Both modes load `.env.local`. Stop with `Ctrl+C`.

Both modes also start the optional Breeze cash-market worker. With no Breeze credentials it waits quietly; after the API key, secret, and daily token are saved under Settings > ICICI Breeze market data, it connects within 30 seconds. Set `BREEZE_WORKER_DISABLED=1` only when the feed should remain off.

Relationship to Oracle

- Local and Oracle use separate PostgreSQL databases.
- Never expose Oracle port 5432 or point routine local development directly at it.
- Move data using `pg_dump`/`pg_restore` over SSH when a snapshot is needed.
- Do not run local and Oracle schedulers against the same database.
- When promoting a separate Ubuntu host to production, treat its database as authoritative and take a

backup before replacing it with a local snapshot.

Source document: deploy/oracle/README.md

InvestoGenie on AWS Lightsail or Oracle-compatible Ubuntu

Source and release behavior reconciled 14 November 2026 through deployed revision `f33f256`.

This deployment package runs Next.js, PostgreSQL, Nginx, and the existing Node/Python scheduler on one Ubuntu VM. The package supports AWS Lightsail and Oracle-compatible Ubuntu hosts; confirm actual production placement with the operator. PostgreSQL and port 3000 remain private; only SSH, HTTP, and HTTPS are exposed.

1. Create the VM

For AWS Lightsail, create an Ubuntu instance with a static public IP, SSH key authentication, and enough memory/storage for PostgreSQL, Next.js and ingestion workers.

For Oracle Cloud, use the tenancy home region and create one `VM.Standard.A1.Flex` instance:

- Ubuntu 24.04 aarch64 image
- 2 OCPUs and 12 GB RAM
- 100 GB boot volume
- public subnet and public IPv4 address
- SSH key authentication

Reserve a public IP and assign it to the VM. This is required if a market-data provider allowlists a fixed source IP. In the subnet security list, allow:

- TCP 22 from your own public IP/CIDR
- TCP 80 from `0.0.0.0/0`
- TCP 443 from `0.0.0.0/0`

Do not expose PostgreSQL 5432 or Next.js 3000.

2. Publish the application revision

The VM clones `https://github.com/abrarakhan/investogenie.git`. Commit and push the exact revision to deploy before bootstrapping the VM. Never commit `.env.local`, database dumps, CAS PDFs, API keys, or passwords.

3. Bootstrap Ubuntu

Copy this directory to the VM if the deployment files are not pushed yet, then:

```
ssh ubuntu@YOUR_RESERVED_IP
sudo DOMAIN=investogenie.example.com \
  REPO_URL=https://github.com/abrarakhan/investogenie.git \
  bash deploy/oracle/bootstrap-ubuntu.sh
```

The script stores generated database and application secrets in the root-owned environment file. Edit it before starting the service:

```
sudoedit /etc/investogenie/investogenie.env
```

When restoring the local database, copy the exact local `CREDENTIAL_ENCRYPTION_KEY`; otherwise existing encrypted SMTP/API credentials cannot be decrypted. Set `NEXT_PUBLIC_APP_URL` to the final HTTPS URL.

4. Transfer the current database

On the Mac:

```
cd /path/to/investogenie
pg_dump --format=custom --compress=6 --no-owner --no-acl \
  --file="$HOME/Desktop/investogenie.dump" investogenie
scp "$HOME/Desktop/investogenie.dump" ubuntu@YOUR_RESERVED_IP:/tmp/
```

On the VM, before the first application start:

```
sudo systemctl stop investogenie 2>/dev/null || true
sudo -u postgres dropdb --if-exists investogenie
sudo -u postgres createdb --owner=investogenie investogenie
sudo -u postgres pg_restore --no-owner --no-acl \
  --role=investogenie --dbname=investogenie /tmp/investogenie.dump
rm /tmp/investogenie.dump
```

The dump contains the source database's schema and market data. Verify its actual migration level; a restore does not automatically add schema changes newer than the dump.

5. Build, start, and enable HTTPS

```
sudo -u investogenie bash -lc \
  'set -a; source /etc/investogenie/investogenie.env; set +a; cd /opt/investogenie/app && npm run build'
sudo systemctl enable --now investogenie
sudo journalctl -u investogenie -f
```

Point the domain's A record at the reserved IP. Once DNS resolves:

```
sudo certbot --nginx -d investogenie.example.com
sudo sed -i 's|NEXT_PUBLIC_APP_URL=http:|NEXT_PUBLIC_APP_URL=https:|' \
  /etc/investogenie/investogenie.env
sudo systemctl restart investogenie
```

Verify:

```
curl -I https://investogenie.example.com/login
sudo systemctl status investogenie postgresql nginx
sudo journalctl -u investogenie --since '30 minutes ago' --no-pager
```

6. Releases and backups

Deploy a pushed `main` revision:

```
sudo bash /opt/investogenie/app/deploy/oracle/deploy-release.sh
```

The release script updates the checkout and re-executes the checked-out version once, then installs locked dependencies, builds Next.js, discovers and applies the ordered migrations from `0033_breeze_credentials.sql` onward in filename order (currently through 0077), verifies required production relations, restarts the web scheduler and Breeze services, and checks the local login endpoint.

Install the local PostgreSQL backup job:

```
sudo install -m 0755 /opt/investogenie/app/deploy/oracle/backup-postgres.sh \
```

```
  /usr/local/sbin/investogenie-backup
echo '20 2 * * * root /usr/local/sbin/investogenie-backup' | \
  sudo tee /etc/cron.d/investogenie-backup
```

The local seven-day rotation protects against bad deployments but not VM or volume loss. Also schedule OCI block-volume backups or copy encrypted dumps to off-instance storage.

Operating notes

- The production service intentionally uses `npm start`, not `npm run dev`.
- The wrapper runs the existing startup, quote, OHLCV, fundamentals, macro,

AMFI, backfill, and digest schedules in the same service.

- Conservative worker limits are set for a 2-OCPU VM. Raise them only after

checking `top`, `free -h`, PostgreSQL activity, and provider throttling.

- A reserved public IP is persistent and can be reassigned to a replacement VM.
- Keep port 5432 private. Use an SSH tunnel for database tools:

```
ssh -N -L 5433:127.0.0.1:5432 ubuntu@YOUR_RESERVED_IP
```

Then connect the desktop database tool to `127.0.0.1:5433`.

ICICI Breeze live market data

The optional Breeze worker makes ICICI WebSocket ticks the primary intraday source for up to 750 priority NSE/BSE cash stocks: open ledger trades, active swing signals, then Nifty 500 members. Yahoo/Google still refresh the broader priority trading surfaces every 5 minutes, and Bhavcopy stays enabled for official EOD reconciliation and corporate-action continuity.

1. Allowlist the server's static public IP in the ICICI Breeze app settings.
2. Deploy normally; the release script installs worker dependencies and keeps

`investogenie-breeze.service` running in a credential-waiting state.

3. Open Settings > ICICI Breeze market data in InvestoGenie.
4. Enter the stable API key and secret during first-time setup, then paste the

newly generated session token each trading day. Saving a replacement token makes the worker reconnect automatically within 30 seconds. Credentials are encrypted in PostgreSQL and are never returned to the browser.

5. Follow logs with `sudo journalctl -u investogenie-breeze -f`.

The cash-market data worker does not place, modify, or cancel orders. It only reads market data and writes `BREEZE_LIVE` rows to `latest_quotes` plus the current daily bar to `daily_ohlcv`.

The separate derivatives/OI bridge uses the same Settings credentials but is kept operator-started to avoid opening a second Breeze session unintentionally: `npm run worker:breeze-oi`. It requires populated derivative contract metadata.

Schema and release verification

The release script assumes migrations 0001-0032 already exist. A fresh empty database needs all migration files in filename order before running the service. A restored older database needs its missing migrations applied deliberately. Startup is not a general migration runner.

The script runs `npm ci --include=dev`, production dependency audit, build and prune; installs Python requirements; applies the explicit migrations; checks required relations/columns; and restarts the web and Breeze services. Lint/tests are separate pre-release gates, not steps in this script.

The schema verifier includes corporate results/coverage, Armed plans, pending orders/fill audits, protected execution plans and Gmail reconnect/error columns. Inspect calendar coverage and Gmail connection health after promotion.

The separate Breeze automatic-buying path is disabled by default and requires user acknowledgement and spending limits. Its supported engines are Strong Swing and saved Adaptive Allocation strategies. Ledger pending buy-stops and confirmed fill records do not send broker orders. The read-only cash market worker and broker account synchronization must not be confused with that optional buying path.

Source document: mobile/README.md

InvestoGenie Mobile

Source reconciled 14 November 2026 through deployed revision `f33f256`.

Android/iOS companion using server-ranked Strong Swing, News & AI Swing and Trade Ledger results. The client does not reproduce financial analytics.

Setup

Apply the server's ordered schema, including mobile sessions (0044), push (0045), ledger returns (0046) and candidate alerts (0050), plus later migrations used by the current server.

```
cd mobile
cp .env.example .env
npm ci
npm start
```

Set `EXPO_PUBLIC_API_URL` to the intended HTTPS deployment. Use an appropriate emulator or native development build; push and biometric capabilities require supported native environments, and push registration requires a physical device.

Tokens are random 256-bit values kept in SecureStore; PostgreSQL stores SHA-256 hashes. Sessions expire after 30 days. Sign-out revokes the device session and disables its push registration.

Implemented surfaces

- Login, India/US selection and Strong Swing gate detail.
- Stock Screener (server sort and search), Swing Candidates (strategy filter), Armed Strong Swing

(buy-stop plan and gates, read-only) and Long-Term Candidates (investor filter).

- Server-ranked News & AI candidates with evidence links.
- Ledger purchase creation, edit/delete, partial/final sales and sale correction.
- Server-calculated capital employed, P&L, ROI and XIRR.
- Price profiles, native SVG candles and five-minute foreground refresh.
- Biometric session re-entry when available.
- Opt-in ledger state-change and ready-candidate notifications (Android; iOS push is off for now).

Adaptive Allocation does not have a mobile route in this revision. Armed Swing pending buy-stop orders and fills are recorded on the web terminal. Mobile ledger entries record trades; they do not directly submit broker orders.

Local release builds

`validate:release` checks `EXPO_PUBLIC_API_URL` (from the environment or `mobile/.env`) is HTTPS. Both build scripts run it first, then regenerate the native project with `expo prebuild --clean`; the generated `ios/` and `android/` folders are not committed.

- Android: `npm run build:android:release` builds a release APK with Gradle

(`android/app/build/outputs/apk/release/`). Sign it with your release keystore.

- iOS: `npm run build:ios:release` builds the Release configuration with Xcode. It needs full Xcode

(not only the Command Line Tools) selected with `xcode-select`, CocoaPods, and an Apple signing team set in Xcode for a device build.

Trade logging and security

A Strong Swing purchase sends only the candidate's server-held `planId`; the server validates it (same market session, captured within 30 minutes, price at or above the entry and below the target) and reads every level from the plan. Candidates without a plan cannot be logged.

The app locks with biometrics, or the device passcode when no biometrics are enrolled, on launch and after 60 seconds in the background. Opening the app from a notification still requires the unlock. Offline at launch keeps the saved session and offers a retry; only a rejected token signs the device out.

Push alerts are Android-only for now. iOS push is off until direct APNs delivery (or an APNs key for Expo's relay) is set up.

Ledger alerts are generic. Ready-candidate messages include ticker, readiness and engine for Strong Swing, legacy Momentum Ignition and Momentum Ignition V2. Expo receives notification text and push tokens, but no detailed prices, quantities, portfolio values or broker data. Server scans still evaluate and persist candidate status observations when no Expo token is enabled; token registration affects delivery only. During an open India session, Momentum readiness is server-authoritative and requires verified fresh Breeze quote and live OHLCV evidence. See privacy (PRIVACY.md).

Local builds and configuration

The documented distribution path is local Expo Prebuild followed by Gradle/Android Studio or Xcode signing. Set the API environment explicitly before bundling:

```
npm run typecheck
npm run validate:release
# Set EXPO_PUBLIC_API_URL for this build in your shell or local environment first.
npx expo prebuild
```

Then build/sign using the generated native project and platform tools. Prebuild changes native files; review generated configuration and test the resulting binary against the intended server.

Two existing package scripts (`build:android:preview`, `build:production`) still invoke EAS Build. Use them only when cloud building is intentionally chosen. `eas.json` remains committed, and `validate:release` checks HTTPS URLs in its preview/production profiles, bundle identifiers and the notification plugin. It does not validate the environment embedded by a separate local build.

The app has an EAS project ID for Expo push registration. Local building does not remove Expo push-relay dependence. No Expo Updates package is declared.

`app.json` declares version 0.1.1, Android versionCode 2 and iOS buildNumber 2; `package.json` remains 0.1.0. These are source metadata, not evidence of a built or installed app.

Server API

Versioned endpoints under `/api/v1/mobile/` provide auth login/logout, me, Strong Swing, News & AI, ledger and nested sale mutations, asset search, candles and push-token registration.

`GET /api/v1/mobile/screens` returns the tab list and `GET /api/v1/mobile/screens/{key}` returns a server-built screen: cards whose values are already computed and formatted, plus option chips and detail sections. Both apps render these with one generic view (`CardScreen` in `App.tsx`), so a new read-only screen is added in `lib/mobileScreens.ts` on the server without an Android or iOS change. Screens that mutate data (Strong Swing trade logging, the ledger) stay as dedicated views. The server enforces authentication and owns projections/ranking. Verify API compatibility and installed binary behavior on physical devices before distribution.

Source document: mobile/PRIVACY.md

InvestoGenie Mobile Privacy

Source behavior reconciled 14 November 2026 through deployed revision `f33f256`.

InvestoGenie Mobile connects to the owner's configured InvestoGenie server.

- Login credentials are sent to the configured API origin. Use HTTPS for deployed builds.
- The raw session token is stored in OS-backed SecureStore. The server stores its SHA-256 hash,

expiry and device/session metadata.

- Trade, portfolio and detailed candidate data are fetched from authenticated server APIs.
- Ledger notifications contain generic state-change text.
- Ready-candidate notifications include the stock ticker, readiness state and engine name.

Expo push delivery receives that text and the device push token. Detailed prices, quantities, portfolio values and broker data are excluded from these payloads.

- Server candidate evaluation and status-observation storage occur for an account even when it has

no enabled Expo token. Token enrollment controls push delivery, not server-side monitoring.

- Biometric checks use platform authentication; the app does not receive or store biometric

templates. Device fallback may be used where supported.

- Mobile purchase/sale actions record ledger events. They do not directly submit broker orders.

Separately enabled server-side Breeze automation has its own controls.

- External news links open when selected and are subject to the destination's data practices.

Sign-out revokes the active mobile session and disables its registered push token. Existing server-side records follow the deployment owner's retention and backup policy; sign-out does not erase ledger history.

Local native builds avoid uploading source through EAS Build when that path is used, but push delivery still uses Expo. EAS Build scripts remain available in the repository and must be chosen deliberately. See mobile setup (README.md) for configuration limits.